VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018



A PROJECT REPORT ON

"Adaptive AI-Based Plagiarism Detection System"

A Dissertation Submitted in partial fulfillment of the requirement for the degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

SAURAV DAS	**(1RG22CS075)**
AYUSH KUMAR	**(1RG23CS011)**
DEEPAK KUMAR V	**(1RG23CS020)**

Under The Guidance of

Mrs. Deepti N N

Assistant Prof.

Dept. of CSE

RGIT, Bengaluru-32



Department of Computer Science & Engineering

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

Cholanagar, R.T. Nagar Post, Bengaluru-560032

2026-2027

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated To Visvesvaraya Technological University)
Cholanagar, R.T. Nagar Post, Bangalore-560032

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

Phase – II

This is to certify that the Project Report entitled **"Adaptive AI-Based Plagiarism Detection System"** is a Bonafide work carried out by **Saurav Das (1RG22CS075), Ayush Kumar (1RG23CS011), Deepak Kumar V (1RG23CS020)** in partial fulfillment for the award of **Bachelor of Engineering in Computer Science Engineering under Visvesvaraya Technological University, Belagavi,** during the year **2026-2027.** It is certified that all corrections/suggestions given for Internal Assessment have been incorporated in the report. This project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

…….	……	……..
Signature of the Guide	**Signature of the HOD**	**Signature of the Principal**
Mrs. Deepti N N	**Dr. Arudra A**	**Dr. D G Anand**
Asst. Prof, Dept. of CSE	**Associate Prof. and Head of Dept of CSE**	**Principal**
RGIT, Bangalore	**RGIT, Bangalore**	**RGIT,Bangalore**

Name of the Examiners	**Signature with date**
1. ……	……
2. ……	……

VISVESVARAYATECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DECLARATION

We hereby declare that the Project Report entitled **"Adaptive AI-Based Plagiarism Detection System"** submitted to the **Visvesvaraya Technological University, Belagavi** during the academic year **2026-2027**, is record of an original project done by us under the guidance of **Mrs. Deepti N N, Asst.Professor, Department of Computer Science and Engineering, Rajiv Gandhi Institute of Technology, Bengaluru** and this project is submitted in the partial fulfillment of requirements for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering.** The results embodied in this Project report have not been submitted to any other University or Institute for award of any degree.

Saurav Das (1RG22CS075)

Ayush Kumar (1RG23CS011)

Deepak Kumar V (1RG23CS020)

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude and respect to the Rajiv Gandhi Institute of Technology, Bengaluru, for providing us with an opportunity to carry out this project work. We express our gratitude to **Dr. D G Anand**, Principal, RGIT, Bengaluru, for providing the resources and support without which the completion of this project would have been a difficult task.

We extend our sincere thanks to **Dr. Arudra A**, Associate Professor and Head, Department of Computer Science and Engineering, RGIT, Bengaluru, for being a pillar of support and encouraging us in the face of all adversities.

We would also like to express our heartfelt thanks to **Prof. Bhagyashree Wakde** and **Prof. Soniya Komal V**, Project Coordinators, for their constant guidance, encouragement, and support throughout the course of this project.

We would like to acknowledge the thorough guidance and support extended to us by **Mrs. Deepti N N**, Assistant Professor, Department of CSE, RGIT, Bengaluru. Her incessant encouragement and valuable technical support have been of immense help. Her guidance provided us with the environment to enhance our knowledge and skills and to reach greater heights with determination, dedication, and hard work.

We also extend our thanks to the entire faculty and support staff of the Department of Computer Science and Engineering, RGIT, Bengaluru, who have encouraged us throughout the course of our Bachelor's degree.

We would like to thank our families for always being there with full support and for providing us with a safe and encouraging environment to complete our project. We are ever grateful to them for helping us during challenging times.

Lastly, we would like to acknowledge all the valuable insights and support given to us by our friends during the course of this project.

Saurav Das (1RG22CS075)
Ayush Kumar (1RG23CS011)
Deepak Kumar V (1RG23CS020)

ABSTRACT

The widespread availability of online content and text-rewriting tools has made plagiarism easier to commit and harder to detect, while most checkers report only a single similarity percentage that misses reworded text and offers no explanation. This project presents PlagiarismAI, an Adaptive AI-Based Plagiarism Detection System that detects, classifies and explains plagiarism using lexical and semantic similarity analysis. It accepts a suspicious document and reference documents in PDF, DOCX or TXT format and computes four independent scores: exact matching, TF-IDF cosine similarity, n-gram overlap and Sentence-BERT semantic similarity. The pattern of these scores is used to classify the likely plagiarism type, with a confidence value, and an adaptive weighting engine then re-weights the four techniques for the detected type instead of applying one fixed formula. Results are shown through a seven-page Streamlit interface with sentence-level evidence, an analytics dashboard and a downloadable PDF report. In a test run, a document compared with itself scored 100% on all four detectors and was classified as Direct Copy with 99.0% confidence. The system is a decision-support tool that leaves the final judgement to a human reviewer.

CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
TABLE OF CONTENTS	iii
LIST OF FIGURES	vi
LIST OF TABLES	vii

1. TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	**Introduction**	**1**
1.1	Background	1
1.2	Motivation	2
1.3	Objectives of the Project	2
1.4	Scope of the Project	3
1.5	Significance of the Study	4
1.6	Organization of the Report	4
1.7	Project Schedule	5
2	**Literature Review**	**7**
2.1	Introduction	7
2.2	Overview of Plagiarism Detection	7
2.3	Literature Survey	12
2.4	Existing System	14
2.5	Proposed System	15
2.6	Comparison Between Existing and Proposed Systems	17
2.7	Research Gap and Summary of Literature Review	18
3	**System Analysis and Design**	**19**
3.1	Introduction	19
3.2	Analysis of the Problem and Design Goals	19
3.3	System Architecture	19
3.4	Module Description	21
3.5	Data Flow and Use Cases	21
3.6	System Requirements	23
3.7	Input and Output Design	24
3.8	Design of the Adaptive Scoring Logic	25
3.9	Data Handling Design	27
3.10	Summary	28
4	**System Implementation**	**29**
4.1	Introduction	29

4.2	Implementation Approach	29
4.3	Implementation of the Modules	29
4.4	Workflow of the Application	32
4.5	Use Cases	33
4.6	Module Interaction	33
4.7	Data Handling and Security	34
4.8	Summary	34
5	**Technologies Used**	**35**
5.1	Introduction	35
5.2	Python	35
5.3	NLTK and Regular Expressions	35
5.4	scikit-learn	35
5.5	Sentence-Transformers and Sentence-BERT	36
5.6	NumPy	36
5.7	Streamlit	36
5.8	Plotly and Document Parsers	37
5.9	ReportLab	37
5.10	Development Environment	37
5.11	Mathematical Foundations	37
5.12	Alternatives Considered	38
5.13	Summary of Technologies Used	39
6	**Implementation**	**40**
6.1	Introduction	40
6.2	Structure of the Application	40
6.3	Algorithms	40
6.4	Page Control Flow	42
6.5	Complexity and Performance	43
6.6	Summary	43
7	**Testing**	**44**
7.1	Introduction	44
7.2	Test Strategy	44
7.3	Test Environment	44
7.4	Test Cases	44
7.5	Observed Result: Identical Documents	45
7.6	Verification of the Score Calculation	46
7.7	Observations and Limitations of the Testing	46
7.8	Summary	47
8	**Sample Output**	**48**
8.1	Introduction	48
8.2	Sample Document	48
8.3	Home Page	48
8.4	Upload & Analyze Page	49
8.5	Results Page	50

8.6	Adaptive Analysis Page	51
8.7	Sentence Analysis Page	53
8.8	Analytics Page	54
8.9	Report Page and PDF Report	55
8.10	Summary of the Sample Run	58
A	**User Manual**	**60**
A.1	Purpose	60
A.2	Starting the Application	60
A.3	Checking a Document	60
A.4	Reading the Result	60
A.5	Sentence Analysis, Analytics and Reports	61
A.6	Guidance for Reviewers	61
A.7	Troubleshooting	61
B	**Source Code Repository**	**62**
B.1	Repository Details	62
B.2	Repository Structure	62
B.3	Installation and Execution	63
B.4	Running the Tests	63
9	**Conclusion and Future Scope**	**64**
9.1	Conclusion	64
9.2	Limitations	64
9.3	Future Scope	64
9.4	Summary	64
	References	**65**

2. LIST OF FIGURES

Chapter No.	Title	Page No.
1	Figure 1.1: Project Schedule (Gantt Chart)	6
2	Figure 2.1: General Workflow of a Text Similarity System	12
2	Figure 2.2: Conceptual Workflow of the Proposed System	16
3	Figure 3.1: Layered Architecture of the System	20
3	Figure 3.2: Data Flow of the System	22
3	Figure 3.3: Use-Case Diagram	22
3	Figure 3.4: Adaptive Type Classification and Weighting	25
4	Figure 4.1: Implementation Workflow	29
4	Figure 4.2: Interaction of the Engine Functions	33
8	Figure 8.1: Home Page	49
8	Figure 8.2: Upload & Analyze Page	50
8	Figure 8.3: Results Page	51
8	Figure 8.4: Adaptive Analysis Page: Similarity Pattern and Scores	52
8	Figure 8.5: Selected Dynamic Weights	52
8	Figure 8.6: Adaptive vs. Fixed-Weight Scoring and Explanation	53
8	Figure 8.7: Sentence Analysis Page	54
8	Figure 8.8: Analytics Dashboard: Metrics and Method Charts	54
8	Figure 8.9: Analytics Dashboard: Pattern, Distribution and Type Summary	55
8	Figure 8.10: Report Page	56
8	Figure 8.11: PDF Report, Page 1	57
8	Figure 8.12: PDF Report, Page 2	58

3. LIST OF TABLES

Chapter No.	Title	Page No.
2	Table 2.1: Overview of Plagiarism Detection Techniques	10
2	Table 2.2: Overview of Relevant Plagiarism Datasets	11
2	Table 2.3: Literature Survey of Related Research	13
2	Table 2.4: Comparison Between Existing and Proposed Systems	17
3	Table 3.1: Design Goals and the Features that Implement Them	19
3	Table 3.2: Modules of the System	21
3	Table 3.3: System Requirements	24
3	Table 3.4: Inputs of the System	24
3	Table 3.5: Outputs of the System by Page	24
3	Table 3.6: Plagiarism Types and Score Patterns	25
3	Table 3.7: Adaptive Weight Profiles and the Fixed Baseline (%)	26
4	Table 4.1: Use Cases of the System	33
5	Table 5.1: Formulas Used by the Detectors	38
5	Table 5.2: Technology Choices and Alternatives	39
5	Table 5.3: Summary of Technologies	39
6	Table 6.1: Application Pages and Their Roles	40
7	Table 7.1: Test Environment	44
7	Table 7.2: Test Cases	45
7	Table 7.3: Observed Values for the Identical-Document Run	45
7	Table 7.4: Expected Behaviour for Planned Behavioural Tests	46
8	Table 8.1: Summary of the Sample Run	58
A	Table A.1: Elements of the Result Pages	60
A	Table A.2: Common Problems and Solutions	61
B	Table B.1: Details of the Source Code Repository	62
B	Table B.2: Folders and Files of the Repository	62

CHAPTER 1

INTRODUCTION

1.1 Background

Writing is the main way in which students, researchers and professionals show what they know. Reports, assignments, theses, research papers, resumes and project documentation all carry an implicit claim that the ideas and wording belong to the person who submitted them. Plagiarism breaks this claim. It is the use of another person's words, ideas or results without proper acknowledgement, and it damages originality, fairness and trust in academic and professional work [1].

The growth of the internet has made plagiarism easier. Large volumes of articles, research papers, source code, project reports and open repositories are one search away, and text can be copied, pasted and lightly edited in seconds. Online paraphrasing tools and generative AI systems make the problem harder still, because a copied passage can now be rewritten so thoroughly that no sequence of words from the original survives. A reviewer who relies on memory or a manual search cannot realistically check every submission against everything that has been published.

Software tools for plagiarism detection were developed to reduce this burden. The earliest approaches compared documents by looking for identical sequences of characters or words, and later approaches added document fingerprinting, vector space models and similarity measures such as TF-IDF with cosine similarity [6], [11]. These methods work well when text has been copied verbatim or with minor edits. They work poorly when the meaning is preserved but the wording is changed, which is exactly the kind of plagiarism that is hardest for a human reviewer to notice [2], [3].

Advances in Natural Language Processing (NLP) now make it possible to compare texts by meaning as well as by surface form. Pre-trained transformer models such as BERT [8] and the sentence-embedding models built on them, such as Sentence-BERT [7], convert a sentence into a numerical vector so that sentences with similar meaning lie close together in vector space. Combining such semantic comparison with classical lexical techniques gives a detector that can respond to both copied and reworded text.

This project, the **Adaptive AI-Based Plagiarism Detection System**, builds on this idea. Instead of producing one similarity percentage from one technique, the system runs four complementary techniques, uses the pattern of their scores to classify the likely type of

plagiarism, adapts the weight given to each technique to that type, and explains the result to the user through per-technique scores, matched sentences and a downloadable report.

1.2 Motivation

Several observations motivated this work. First, manual checking is slow and unreliable. A faculty member who evaluates dozens of reports cannot search for the source of every suspicious paragraph, and subtle reuse is easily missed. An automated first check that points the reviewer to the passages worth examining saves time and makes the review more consistent.

Second, the plagiarism that matters most today is often not verbatim. Studies of plagiarism detection note that the difficulty of a case grows with the amount of rewording: copied text is easy to find, lightly revised text is harder, and heavily revised or idea-level plagiarism is hardest [3], [5]. A tool that applies one technique with one fixed threshold to all of these cases will be tuned for one of them and weak on the others.

Third, a bare percentage is difficult to act upon. When a checker reports “42% similarity” without showing which technique produced the number or which sentences matched, the reviewer cannot tell whether the figure reflects copied paragraphs, common technical vocabulary or a properly quoted passage. Such opacity can lead to unfair accusations as easily as to missed cases. Explanations that a reviewer can read and verify are therefore an important requirement.

Fourth, plagiarism is not limited to formal reports. Students and job applicants also reuse text in resumes, portfolio descriptions, GitHub project README files and answers given in project viva examinations. These documents are short, domain-specific and full of shared technical vocabulary, and a detector that is tuned only for long essays tends to behave poorly on them. This is the reason the system is designed to adapt its behaviour to the kind of similarity it observes.

Finally, the project offered an opportunity to apply the NLP, machine-learning and software-engineering topics studied during the course in a single practical system, covering document parsing, text processing, feature extraction, similarity measures, a multi-page web interface, interactive analytics and report generation.

1.3 Objectives of the Project

The main aim of the project is to design and implement a plagiarism detection system that recognises both copied and reworded text and explains its findings. The specific objectives are:

1. To study existing plagiarism detection techniques and identify the types of plagiarism that each technique detects well or poorly.
2. To build a text-preprocessing pipeline that cleans and normalises documents using tokenization, stopword removal and stemming.
3. To implement four similarity detectors: verbatim n-gram matching, TF-IDF with cosine similarity, word n-gram containment and Sentence-BERT semantic similarity.
4. To classify the likely type of plagiarism (direct copy, partial copy, paraphrased, structural, mixed, or low/no plagiarism) from the pattern of detector scores.
5. To adapt the weight given to each detector according to the detected type and to compute a final similarity score.
6. To present the result transparently, with per-technique scores, the applied weights, highlighted matching sentences and a verdict.
7. To develop a multi-page web interface in which a user can upload a suspicious document and one or more reference documents (PDF, DOCX or TXT), or use bundled samples, and view the analysis page by page.
8. To provide explainable output: a plain-language explanation, sentence-level evidence, an analytics dashboard, a comparison of adaptive and fixed-weight scoring, and a downloadable PDF report.
9. To test the system with test cases and sample documents and to report the observed behaviour honestly, including its limitations.

1.4 Scope of the Project

The scope of the present work is defined as follows.

- **Text comparison:** the system compares a suspicious document against one or more reference documents supplied by the user or taken from the bundled sample references. It does not crawl the public internet.
- **Document types:** PDF, DOCX and plain-text (TXT) files of up to 200 MB each can be uploaded, or bundled sample documents can be used. The design targets academic reports and assignments, resumes, portfolio descriptions, GitHub project descriptions and viva answers.
- **Language:** the processing pipeline and the default embedding model are intended for English text.

- **Techniques:** four similarity techniques are combined: exact n-gram matching, TF-IDF cosine similarity, n-gram containment and Sentence-BERT semantic similarity.
- **Explainability:** the output includes the individual scores, the detected plagiarism type, the weight profile used, the matching sentence pairs and a PDF report.
- **Result handling:** the results of the latest analysis are held in the browser session and shared by all pages of the application. No persistent database is used in this version; a stored history is listed as future scope.
- **Out of scope:** source-code plagiarism, cross-language plagiarism, image-based plagiarism and automatic detection of AI-generated text are not covered in this phase. They are discussed under future scope in Chapter 9.

> **Important:** the system is a decision-support tool. A high similarity score indicates that a document deserves closer review. It is not proof of misconduct, because properly quoted passages, standard definitions and common technical phrases also produce similarity.

1.5 Significance of the Study

The project is significant from technical, academic and practical points of view.

From a technical point of view, it demonstrates how classical lexical methods and modern semantic embeddings complement each other. Lexical methods are fast, precise and easy to explain, but they cannot see through rewording. Embedding-based methods capture meaning but can give moderate scores to texts that merely discuss the same topic. Using the pattern of both kinds of scores to decide which one to trust is the central idea of the adaptive design.

From an academic point of view, the system can help institutions give faster and more consistent first-level screening of submissions. Because the report shows why a document was flagged, faculty can verify the evidence before taking any decision, and students can use the same tool to check their own work before submission.

From a practical point of view, the project combines document parsing, text processing, machine-learning libraries, an interactive web interface and PDF generation into one complete application, which is a useful demonstration of end-to-end software development. The same engine can later be exposed through an API or integrated with a learning management system.

1.6 Organization of the Report

The rest of this report is organised as follows.

- **Chapter 2 (Literature Review)** surveys plagiarism types, detection techniques, datasets and related research, describes the existing and proposed systems and states the research gap.
- **Chapter 3 (System Analysis and Design)** presents the architecture, modules, requirements, input and output design, the adaptive scoring logic and the data flow.
- **Chapter 4 (System Implementation)** describes the tools, the implementation of each module, the workflow and use cases, data handling and security considerations.
- **Chapter 5 (Technologies Used)** explains Python, NLTK, scikit-learn, Sentence-Transformers, Streamlit, Plotly and ReportLab.
- **Chapter 6 (Implementation)** gives the main algorithms of the system in pseudo-code and the structure of the application pages.
- **Chapter 7 (Testing)** describes the test strategy, test environment, test cases and the results observed so far.
- **Chapter 8 (Sample Output)** shows the application pages and the generated PDF report for the sample run.
- **Appendix A** is a short user manual for the application.
- **Appendix B** describes the GitHub repository and how to run the source code.
- **Chapter 9 (Conclusion and Future Scope)** summarises the work, lists its limitations and proposes future enhancements.

1.7 Project Schedule

The project was planned as a sequence of activities, shown as a Gantt chart in Figure 1.1. It starts with topic selection, problem identification and a literature survey of traditional plagiarism detection, NLP-based techniques and machine-learning and deep-learning models. Requirement analysis then identifies the software tools, libraries and data needed. Sample documents are collected and preprocessed using tokenization, stemming, stopword removal and normalisation. In the design phase the architecture, workflow, page structure and module interactions are planned. The implementation phase then builds the text-similarity techniques (exact matching, TF-IDF with cosine similarity and n-gram containment), the semantic module based on Sentence-BERT, the plagiarism-type classifier, the adaptive weighting logic, the multi-page Streamlit interface with its analytics, and the PDF report. The system is finally tested, the results are analysed, and the technical documentation, report and presentation are prepared.

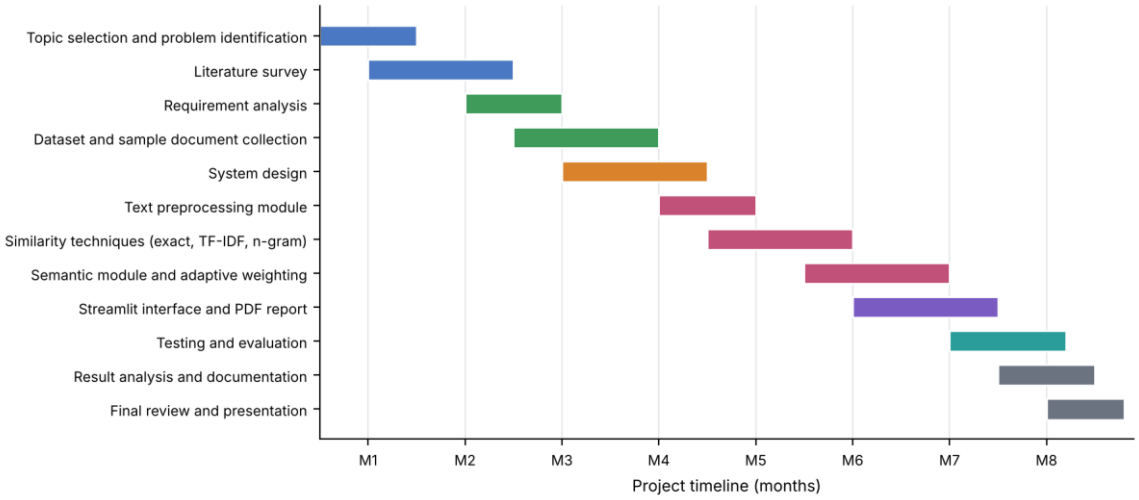


Figure 1.1: Project Schedule (Gantt Chart)

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Plagiarism detection is one of the oldest applications of text similarity. Research in this area covers how plagiarism is defined and classified, which textual features can reveal reuse, how documents are compared at scale, and how detection systems should be evaluated. This chapter reviews the concepts and research that are relevant to the project. It first gives an overview of plagiarism types and detection techniques, then surveys representative studies, then describes the existing approach and the proposed system, compares the two and finally states the research gap that the project addresses.

The review is organised around one practical question: given a submitted document and a reference document, how can a system decide whether the submission reuses the reference, and how can it justify its decision to a human reviewer? Different families of techniques answer this question for different kinds of reuse, and the survey shows that no single technique answers it for all of them.

2.2 Overview of Plagiarism Detection

2.2.1 Plagiarism and Its Types

Plagiarism is commonly described as the use of another person's words, ideas or results without giving credit. Maurer et al. [1] survey the forms that plagiarism takes, from copying text word for word to reusing ideas, translating text from another language and, in programming courses, copying source code. Alzahrani et al. [2] relate these forms to linguistic patterns and textual features and discuss how the detection method should be chosen to suit the form of reuse.

For text, the forms of plagiarism that matter to this project differ mainly in how much the wording has been changed:

- **Direct copy:** passages are copied word for word or almost word for word, sometimes with quotation marks or citations removed.
- **Partial copy:** some sentences or paragraphs are taken from the source, with minor edits such as replaced words or small insertions, while the remaining text is original.
- **Paraphrased:** sentences are rewritten so that few exact sequences remain, but the vocabulary and the sentence-level meaning are still close to the source.

- **Structural:** the organisation of the source is reused, for example by reordering, merging or splitting sentences, so that the same words or ideas appear in a different arrangement and exact sequences are broken.
- **Mixed:** several of the above occur in one document, for example some sentences copied directly and others paraphrased.
- **Low / no plagiarism:** the texts share only common vocabulary or standard phrases, or are unrelated.

This graded view is consistent with the way Clough and Stevenson [5] built a corpus of plagiarised short answers, in which answers were written as near copies, light revisions or heavy revisions of a source text, alongside non-plagiarised answers. The type taxonomy used by the proposed system (Section 2.5) follows the same idea and extends it with structural, mixed and low/no-plagiarism cases, because the kind and amount of rewording determine which detection technique is reliable.

2.2.2 Natural Language Processing for Text Similarity

Natural Language Processing (NLP) provides the tools that turn raw text into a form that can be compared. The basic steps are tokenization (splitting text into words or sentences), case normalisation, removal of stopwords such as “the” and “is” that carry little meaning, and stemming or lemmatization, which reduce related word forms such as “plagiarised” and “plagiarising” to a common root [16]. After preprocessing, texts can be represented as sets of words, sets of word sequences (n-grams), weighted term vectors or dense embedding vectors.

Early statistical representations such as the bag of words ignore word order and treat every word as independent. Term weighting schemes such as TF-IDF improve on this by giving a higher weight to terms that are frequent in a document but rare across documents [6]. Neural representations went further. Word embedding methods such as Word2Vec [9] and GloVe [10] learn a vector for each word from co-occurrence patterns in large corpora, so that words used in similar contexts receive similar vectors. Contextual models based on the Transformer architecture [15], such as BERT [8], produce vectors that depend on the surrounding sentence. Sentence-BERT [7] fine-tunes BERT-style networks so that whole sentences are mapped to fixed-size vectors that can be compared directly with cosine similarity.

The practical consequence is that lexical representations measure how much the surface wording of two texts overlaps, while embedding representations measure how close their

meanings are. Plagiarism detection needs both views, because copied text is best identified by the first and reworded text by the second.

2.2.3 Detection Techniques Used in Plagiarism Detection

Foltynek et al. [3] review academic plagiarism detection methods and show that approaches range from lexical and syntactic comparison to semantic and idea-based analysis, with detection getting harder as the amount of rewording increases. The following techniques are most relevant to this project.

String matching and fingerprinting. Exact comparison looks for identical character or word sequences. To avoid comparing every pair of positions, fingerprinting methods select a subset of hashes from each document. The winnowing algorithm of Schleimer et al. [11], on which the MOSS system is based, guarantees that any shared substring longer than a chosen length produces at least one shared fingerprint while keeping the number of stored hashes small.

N-gram shingling and containment. Broder [12] represents a document as a set of word shingles (n-grams) and defines resemblance and containment between sets. Containment, the fraction of one document's shingles that appear in another, is a natural measure for plagiarism because a short submission may be entirely contained inside a long source even when the two documents are very different in size.

TF-IDF with cosine similarity. Each document becomes a vector of TF-IDF weights, and the cosine of the angle between two vectors measures their similarity [6]. This captures vocabulary overlap while down-weighting common words, and it tolerates reordering of words and sentences. It does not capture synonyms.

Embedding-based semantic similarity. Sentence embeddings place sentences with similar meaning close together, so a paraphrase can score highly even when it shares few words with the source [7]. Reimers and Gurevych report that Sentence-BERT reduces the time needed to find the most similar pair in a collection of 10,000 sentences from about 65 hours with BERT to about 5 seconds, which makes sentence-level comparison practical. The all-MiniLM family of sentence-transformer models is built on the compact MiniLM architecture [17] and offers a good balance between speed and quality for this purpose.

Table 2.1: Overview of Plagiarism Detection Techniques

Technique	Main principle	Potential advantage	Important limitation
Exact matching / fingerprinting [11]	Finds identical word or character sequences, often through selected hashes	Precise and fast, easy to explain	Fails when words are replaced or reordered
N-gram containment [12]	Measures the share of one text's word n-grams found in another	Good for partial copying and unequal document lengths	Sensitive to synonym substitution; n must be chosen
TF-IDF with cosine similarity [6]	Compares weighted term vectors by the angle between them	Tolerates reordering; down-weights common words	Does not understand synonyms or meaning
Word embeddings [9], [10]	Maps words to vectors learned from co-occurrence	Captures word-level similarity	One vector per word regardless of context
Sentence-BERT [7]	Maps sentences to vectors using fine-tuned transformer networks	Detects paraphrase and meaning-level similarity	Heavier to run; can give moderate scores to merely related topics

2.2.4 Datasets and Corpora for Plagiarism Detection

Evaluating plagiarism detectors requires documents in which the amount and kind of reuse is known. Two corpora are widely cited in the literature, and the project also uses a small set of its own sample pairs for demonstration.

The PAN plagiarism detection competition [4] provided a large corpus in which plagiarism cases were inserted artificially into documents, with different degrees of obfuscation, and compared systems on a common footing. Potthast et al. [18] later proposed a general evaluation framework for plagiarism detection, defining measures that take into account both whether a case was found and how precisely its boundaries were located. The corpus of Clough and Stevenson [5] is much smaller, but it contains short answers to questions written at the levels of near copy, light revision and heavy revision, which makes it useful for studying how detection scores fall as rewording increases.

Table 2.2: Overview of Relevant Plagiarism Datasets

Feature	PAN competition corpus [4]	Clough and Stevenson corpus [5]	Project sample pairs
Main purpose	Evaluation of external and intrinsic plagiarism detection	Study of plagiarism in short answers	Demonstration and unit testing of the system
Plagiarism cases	Artificially inserted, with varying obfuscation	Written by participants at defined levels of rewording	Bundled sample documents and hand-made test pairs
Document length	Long documents	Short answers	Short documents of a few sentences
Levels of reuse	Several obfuscation strategies	Near copy, light revision, heavy revision, non-plagiarised	Direct copy, partial copy, paraphrase, unrelated
Availability in this work	Described in literature; not used for scoring	Described in literature; not used for scoring	Used (Chapters 7 and 8)

Note: the scores reported in this project are measured on the project's own sample documents. They illustrate how the system behaves and must not be compared with figures published for the PAN or Clough–Stevenson corpora, which were obtained on different data.

2.2.5 Importance of Preprocessing and Evaluation

Preprocessing has a large effect on similarity scores. Without stopword removal, two unrelated English documents share many common words and TF-IDF vectors are dominated by function words. Without case normalisation and punctuation removal, "Model," and "model" count as different tokens. Without stemming, "detects" and "detection" are treated as different words. At the same time, aggressive preprocessing can remove information that is useful for exact matching, such as word order and punctuation. The proposed system therefore applies stemming and stopword removal for the vector-based techniques but runs its verbatim detector on the lightly normalised original word sequence.

Evaluation of a detector should consider both kinds of error. A false positive marks original work as plagiarised and can harm an innocent author. A false negative lets plagiarism pass. Precision (the share of flagged cases that are truly plagiarised), recall (the share of true cases that are flagged) and the F1-score (their harmonic mean) summarise these errors, and a confusion matrix shows which classes are confused with each other. For a system that also assigns a plagiarism type, the confusion matrix over types is informative because it shows, for example, whether paraphrases are mistaken for original text.

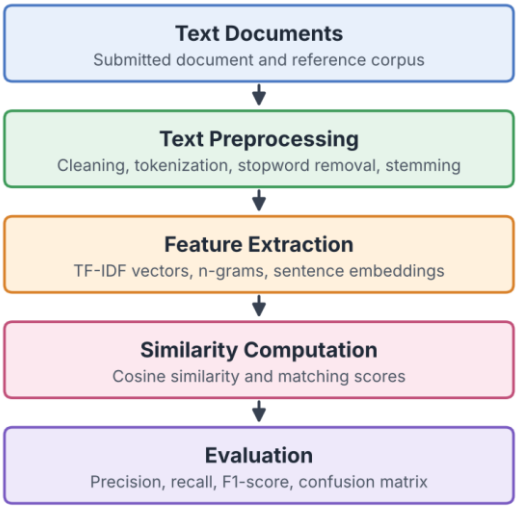


Figure 2.1: General Workflow of a Text Similarity System

Figure 2.1 summarises the common workflow: documents are cleaned, converted to features, compared with a similarity measure and evaluated. The proposed system follows this workflow but repeats the feature and similarity stages with four techniques and adds a classification and weighting stage before the final score.

2.3 Literature Survey

The literature survey groups the studies that influenced the design of the system into three themes: surveys and taxonomies of plagiarism, lexical and fingerprinting approaches, and semantic and embedding-based approaches. Table 2.3 summarises the studies.

2.3.1 Surveys and Taxonomies of Plagiarism Detection

Maurer et al. [1] present an early and widely cited survey of plagiarism and of the methods and tools used against it. They discuss the forms of plagiarism, the services available to detect it and the limitations of the tools of that time, and conclude that a tool can only point to suspicious passages while the final judgement must remain with a human. This view of detection as decision support is adopted in this project.

Alzahrani et al. [2] review the linguistic patterns and textual features that distinguish plagiarised from original text and the methods that exploit them, including fingerprinting, term occurrence analysis and stylometry. Lukashenko et al. [13] compare computer-based plagiarism detection methods and tools. Foltynek et al. [3] provide a more recent systematic review, covering lexical, syntactic, semantic and idea-based approaches, and note that heavily

disguised and idea-level plagiarism remain the most difficult cases. Together, these surveys motivate the combination of several techniques rather than reliance on one.

2.3.2 Research on Lexical and Fingerprinting Approaches

Salton and Buckley [6] studied term-weighting approaches in text retrieval and established TF-IDF weighting, combined with cosine normalisation, as a strong and simple basis for measuring document similarity. Broder [12] introduced shingling to estimate the resemblance and containment of documents, and Schleimer et al. [11] developed winnowing, which selects document fingerprints so that long matches are guaranteed to be found while storage stays small. These methods are efficient and their outputs are easy to explain, because the matching passages can be shown to the user. Their weakness, noted in the surveys above, is that they depend on the surface wording.

2.3.3 Research on Semantic and Embedding-Based Approaches

Mikolov et al. [9] and Pennington et al. [10] introduced word embeddings that capture semantic relations between words. Devlin et al. [8] showed that a deep bidirectional Transformer encoder, pre-trained on large corpora, can be fine-tuned for many language-understanding tasks, and Vaswani et al. [15] described the attention-based architecture on which it is built. Reimers and Gurevych [7] adapted BERT to produce sentence embeddings that are directly comparable by cosine similarity, and Wang et al. [17] proposed compact MiniLM models that retain much of the quality of larger models at a fraction of the cost. These results make it practical to compare submitted and reference text at sentence level on ordinary hardware, which is the basis of the semantic detector in this project.

Table 2.3: Literature Survey of Related Research

Study	Area and method	Main contribution	Relevance and limitation
Maurer, Kappe and Zaka (2006) [1]	Survey of plagiarism and detection services	Describes forms of plagiarism and the capabilities of detection tools	Establishes detection as decision support; predates embedding methods
Alzahrani, Salim and Abraham (2012) [2]	Review of linguistic patterns, features and methods	Relates textual features and plagiarism types to detection methods	Supports choosing techniques by plagiarism type
Lukashenko, Graudina and Grundspenkis (2007) [13]	Overview of computer-based detection methods and tools	Compares families of methods and available tools	Descriptive; no semantic methods
Foltynek, Meuschke and Gipp (2019) [3]	Systematic literature review of academic plagiarism detection	Organises methods from lexical to idea-based and identifies open problems	Shows paraphrase and idea-level reuse remain hard
Salton and Buckley (1988) [6]	Term weighting for text retrieval	Establishes TF-IDF with cosine normalisation	Lexical only; used here as one detector

Study	Area and method	Main contribution	Relevance and limitation
Broder (1997) [12]	Shingling; resemblance and containment of documents	Set-based measures suited to partial copying	Sensitive to word substitution; used here as n-gram containment
Schleimer, Wilkerson and Aiken (2003) [11]	Winnowing document fingerprinting	Guarantees detection of long shared substrings with few hashes	Exact matches only
Potthast et al. (2009) [4]	PAN competition on plagiarism detection	Common corpus and comparison of detection systems	Artificial cases; different from student documents
Clough and Stevenson (2011) [5]	Corpus of plagiarised short answers	Cases written at levels of near copy, light and heavy revision	Basis for the type taxonomy; small corpus
Reimers and Gurevych (2019) [7]	Sentence-BERT sentence embeddings	Fast, cosine-comparable sentence vectors	Basis of the semantic detector; related topics may score moderately

2.4 Existing System

Most plagiarism checkers that are widely available to students and institutions follow the same broad design: the submitted document is compared with a very large index of web pages, publications and previously submitted work, and the output is an overall similarity percentage together with a list of matching sources. The comparison is dominated by lexical matching of word sequences. This section describes the common approaches and their limitations.

2.4.1 Manual and Exact-Match Checking

The simplest method is manual checking, in which a reviewer reads the document and searches suspicious sentences on the web. This is slow, depends on the reviewer's memory and attention, and does not scale to large batches. Simple software tools automate part of this by comparing documents for identical sequences of words or characters. They are precise for copied text and produce a result that is easy to explain, but they offer little help for reworded text.

2.4.2 Vector-Space and Machine-Learning-Based Systems

More advanced systems represent documents as TF-IDF vectors and compare them with cosine similarity, or train classifiers on features extracted from document pairs. Systems that use pre-trained embeddings can recognise some paraphrases. However, most such systems use a single fixed combination of methods and thresholds for every input, so the same weight is given to the lexical and the semantic evidence regardless of what kind of similarity is actually present.

2.4.3 Limitations of Existing Approaches

The literature and the practical use of existing tools show the following limitations.

1. **Weak detection of paraphrase:** systems that depend on exact or near-exact matching miss reworded text [2], [3].

2. **One fixed strategy:** a single technique and one set of thresholds is applied to all inputs, so the system is tuned for one type of reuse and weaker on the others.
3. **Opaque results:** many tools present only an overall percentage, from which a reviewer cannot tell which technique or passage caused the score.
4. **Poor fit for short and technical documents:** resumes, portfolio entries and viva answers are short and share standard technical vocabulary, which inflates lexical similarity.
5. **Dependence on external services:** web-scale checkers require uploading documents to a third-party service, which raises privacy and cost concerns for institutions and individuals.
6. **Limited feedback to the author:** students and applicants cannot easily see which sentences are problematic or how to correct them.

2.5 Proposed System

The proposed Adaptive AI-Based Plagiarism Detection System is a web-based application that compares a submitted document with reference text using four complementary techniques, classifies the type of similarity it finds and adapts the way the scores are combined to that type. Figure 2.2 shows the conceptual workflow.

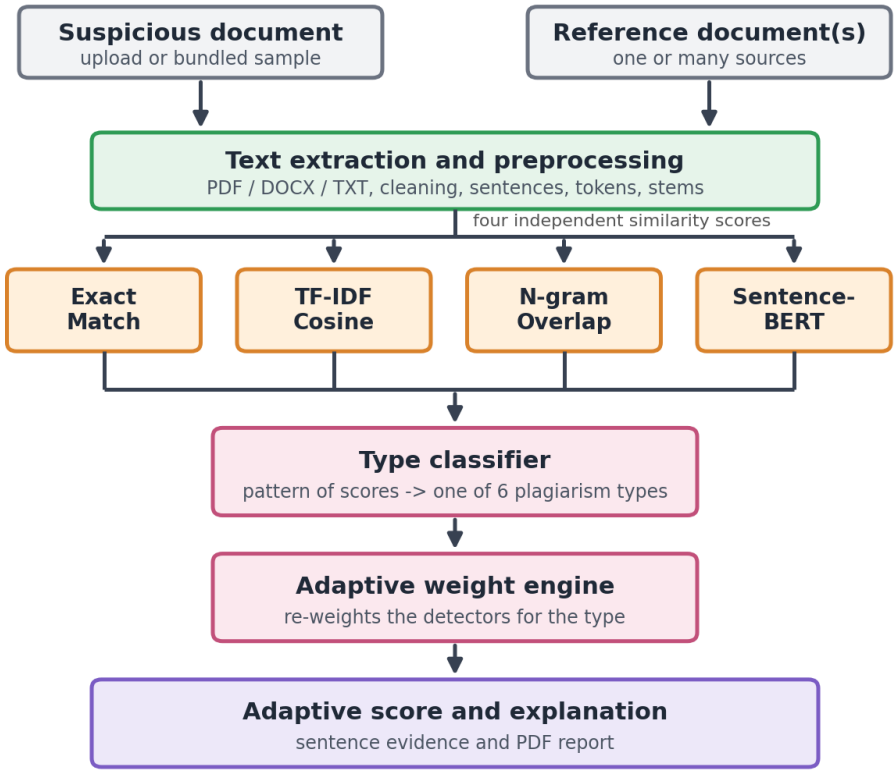


Figure 2.2: Conceptual Workflow of the Proposed System

2.5.1 Multi-Technique Similarity Detection

The system runs four detectors on every comparison. The exact-match detector measures the share of the submitted text covered by verbatim word n-grams. The TF-IDF detector computes the cosine similarity of weighted term vectors. The n-gram detector measures the containment of the submitted text's stemmed word trigrams in the reference. The semantic detector computes the mean best-match cosine similarity between Sentence-BERT embeddings of the sentences of the two texts. Each detector returns a score between 0 and 1.

2.5.2 Plagiarism-Type Classification and Adaptive Weighting

The four scores form a pattern. A high exact-match score together with high n-gram containment indicates a direct copy. A clear lexical signal that covers only part of the text indicates a partial copy. A high semantic score with low exact and n-gram scores indicates a paraphrase, or a structural copy when the TF-IDF score is also high because the same vocabulary has been rearranged. Lexical and semantic evidence together indicate mixed plagiarism, and uniformly low scores indicate low or no plagiarism. The system classifies the pair into one of these six types by rules on the score pattern, reports a confidence, and then selects a weight profile that gives the most weight to the detectors that are reliable for that type. The adaptive score is the weighted sum of the four scores and is shown next to the traditional

fixed-weight score so that the effect of adaptation is visible. Adapting the weights in this way prevents a strong lexical result from being diluted by a low semantic score in the case of a direct copy, and prevents a low lexical score from hiding a paraphrase.

2.5.3 Explainable Results, Web Interface and Reports

The Streamlit interface is organised as seven pages. The user uploads a suspicious document and one or more reference documents (or chooses bundled samples) and runs the analysis. The Results page shows the adaptive score, the detected type, the classification confidence, the most similar source and the number of suspicious sentences, together with a chart of the four detector scores. Further pages show the adaptive weights and explanation, sentence-level evidence, an analytics dashboard and a downloadable PDF report. The latest result is kept in the browser session, so all pages read the same analysis.

2.5.4 Expected Benefits of the Proposed System

- Detection of both copied and reworded content by combining lexical and semantic evidence.
- A result that is explained through scores, type, weights and matching sentences rather than a single percentage.
- Local processing, so documents do not have to be sent to an external service.
- Sentence-level evidence, an analytics dashboard and a downloadable PDF report for record keeping.
- A modular design in which detectors, weight profiles and thresholds can be changed without rewriting the application.

2.6 Comparison Between Existing and Proposed Systems

Table 2.4: Comparison Between Existing and Proposed Systems

Feature	Existing approach	Proposed system
Detection technique	Single technique, mostly exact or near-exact matching	Four techniques: exact, TF-IDF, n-gram containment and Sentence-BERT
Paraphrase detection	Weak or absent	Supported through semantic embeddings
Scoring	One fixed method and fixed thresholds	Weights adapt to the detected plagiarism type
Plagiarism type	Not reported	Classified as direct copy, partial copy, paraphrased, structural, mixed or low/no plagiarism
Explainability	Overall percentage and a list of sources	Per-technique scores, applied weights, matched sentences and verdict

Feature	Existing approach	Proposed system
Short technical documents	Not specifically handled	Designed for resumes, portfolios, project descriptions and viva answers
Data handling	Documents often uploaded to an external service	Processed locally; results are kept only in the user's browser session
Reporting	Vendor-specific reports	PDF report generated with ReportLab
Extensibility	Closed	Modular Python code in which detectors and profiles can be changed

2.7 Research Gap and Summary of Literature Review

2.7.1 Research Gap

The review shows that lexical methods are precise and explainable but blind to rewording, while semantic methods see through rewording but are less precise and can overstate topical similarity [3], [7]. Surveys recommend combining techniques, yet in practice the combination is usually a fixed average or a single threshold. Three gaps follow:

1. There is little support for **choosing how to trust each technique according to the kind of similarity present** in a particular document pair.
2. Many systems **do not explain** which technique and which sentences produced a score, which limits their usefulness as decision support.
3. Tools are rarely tuned for **short, technical documents** such as resumes, portfolio entries, project descriptions and viva answers, which are an increasing source of concern in academic and recruitment settings.

The proposed system addresses these gaps by classifying the plagiarism type from the score pattern, adapting the weights to that type and presenting an explainable result.

2.7.2 Summary of Literature Review

This chapter reviewed the types of plagiarism, the NLP representations used to compare texts, the main detection techniques and the datasets and measures used to evaluate them. The studies surveyed show that detection becomes harder as rewording increases, that lexical techniques and semantic techniques are complementary, and that results should be explained to a human reviewer. The existing approach was described and its limitations listed, and the proposed adaptive system was outlined and compared with it. The next chapter presents the analysis and design of the proposed system.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents the analysis and design of the Adaptive AI-Based Plagiarism Detection System, which is released in the application as **PlagiarismAI v1.0**. It states the design goals, describes the layered architecture and the modules, shows the data flow and the use cases, lists the functional and non-functional requirements, describes the input and output design of the seven application pages and, most importantly, specifies the adaptive scoring logic: how the type of plagiarism is classified, which weights are applied and how the final score, the alert and the explanation are produced.

3.2 Analysis of the Problem and Design Goals

The analysis of existing tools in Chapter 2 shows that a useful detector must do more than report one percentage. A reviewer needs to know how much of a document matches, what kind of match it is, which sentences are responsible and why the system reached its conclusion. Table 3.1 translates these needs into design goals and the features that implement them.

Table 3.1: Design Goals and the Features that Implement Them

Design goal	Reason	Feature in PlagiarismAI
Detect copied and reworded text	Lexical methods miss paraphrase; semantic methods blur exact reuse	Four independent detectors: exact match, TF-IDF, n-gram overlap and Sentence-BERT
Know the kind of plagiarism	The reliable technique depends on how much the text was reworded	Classifier with six types and a confidence value
Avoid one formula for every document	A fixed average dilutes strong evidence	Adaptive weighting engine with a profile for each type
Show the benefit of adaptation	The reviewer should see what adaptation changes	Adaptive versus fixed-weight comparison (Module 10)
Explain every result	A bare percentage cannot be verified	Plain-language explanation and sentence-level evidence
Accept real documents	Submissions are PDF or Word files	PDF, DOCX and TXT upload, plus several reference documents
Support review and record keeping	Evidence must be inspected and filed	Analytics dashboard and downloadable PDF report

3.3 System Architecture

The system follows a layered architecture, shown in Figure 3.1. Each layer has one responsibility and communicates only with the layer next to it, so a layer can be changed, for example a different embedding model, without changing the others.

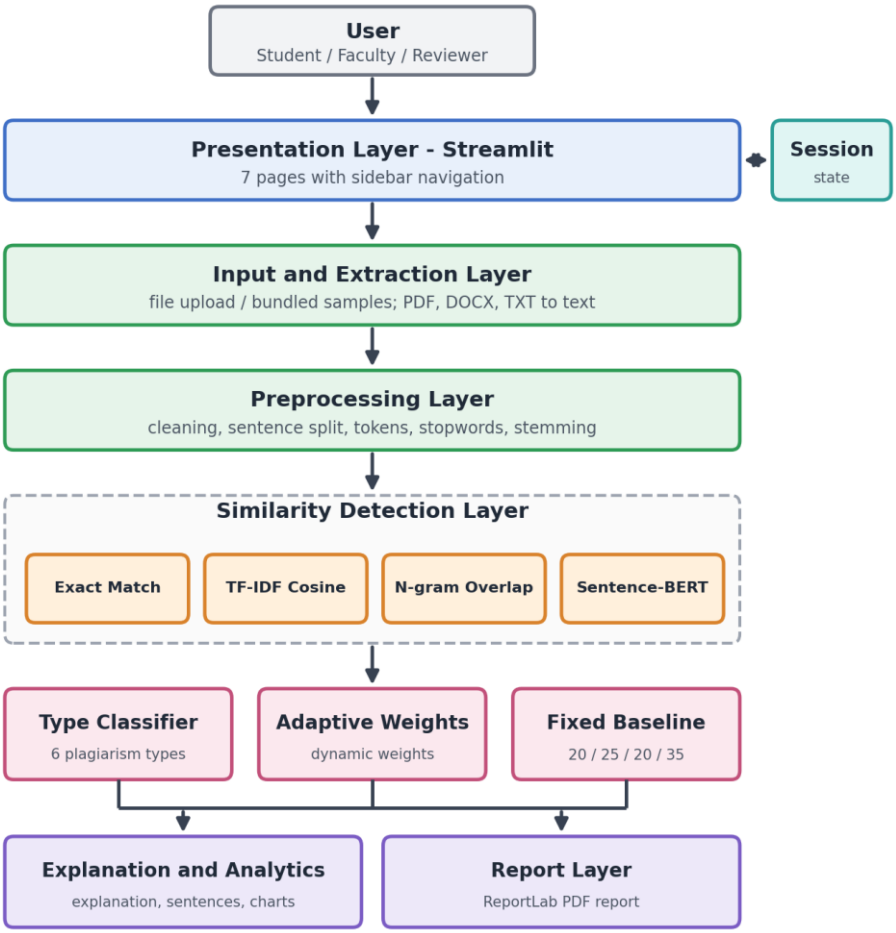


Figure 3.1: Layered Architecture of the System

- **Presentation layer (Streamlit).** A sidebar navigation menu opens seven pages: Home, Upload & Analyze, Results, Adaptive Analysis, Sentence Analysis, Analytics and Report. The latest analysis result is kept in the session state so that every page reads the same data.
- **Input and extraction layer.** Receives the suspicious document and the reference documents, either as uploaded files or as bundled samples, and converts PDF, DOCX and TXT files into plain text.
- **Preprocessing layer.** Cleans and normalises the text, splits it into sentences and tokens and prepares the stemmed, stopword-free form used by the vector-based detectors.
- **Similarity detection layer.** Runs the four detectors independently and returns four scores between 0 and 1.
- **Adaptation layer.** The type classifier reads the pattern of the four scores, the adaptive weight engine selects the weight profile for that type, and the fixed-weight baseline provides the traditional score for comparison.

- **Explanation and analytics layer.** Produces the explanation text, the sentence-level evidence and the Plotly charts.
- **Report layer.** Builds the downloadable PDF report with ReportLab from the same result object.

3.4 Module Description

The detection engine is divided into ten numbered modules. The Adaptive Analysis page names the last of them, Module 10, explicitly. The remaining functions of the application, explanation, sentence analysis, analytics and reporting, build on the output of these modules. Table 3.2 summarises the modules.

Table 3.2: Modules of the System

No.	Module	Function	Output
1	Document input	Accepts an uploaded file or a bundled sample for the suspicious document, and uploaded files or a bundled reference for the references; checks the file type and size	Document objects
2	Text extraction	Reads TXT, DOCX and PDF files and returns plain text	Raw text
3	Preprocessing	Lower-casing, punctuation and stopword removal, stemming, sentence and token splitting	Clean text, sentences, tokens
4	Exact matching	Measures the share of the suspicious text covered by verbatim word sequences found in the reference	Exact score
5	TF-IDF similarity	Cosine similarity of TF-IDF vectors of the two documents	TF-IDF score
6	N-gram overlap	Containment of the suspicious text's word n-grams in the reference	N-gram score
7	Semantic similarity	Sentence-BERT embeddings compared by cosine similarity	Semantic score
8	Type classification	Maps the four scores to one of six plagiarism types with a confidence	Type, confidence
9	Adaptive weighting	Selects the weight profile of the detected type and computes the adaptive score	Weights, adaptive score
10	Adaptive vs. fixed-weight scoring	Computes the traditional fixed-weight score and the difference from the adaptive score	Fixed score, difference
—	Explanation and sentence analysis	Builds the explanation and compares every suspicious sentence with the reference sentences	Explanation, flagged sentences
—	Analytics and report	Draws the dashboard charts and generates the PDF report	Charts, PDF

3.5 Data Flow and Use Cases

Figure 3.2 shows the data flow. The user supplies documents, which are validated, converted to text and preprocessed. The four detectors produce scores, the classifier and weight engine

turn them into a type, weights and a final score, and the explanation, charts and report are produced from that result. The result is held in session state, which the later pages read.

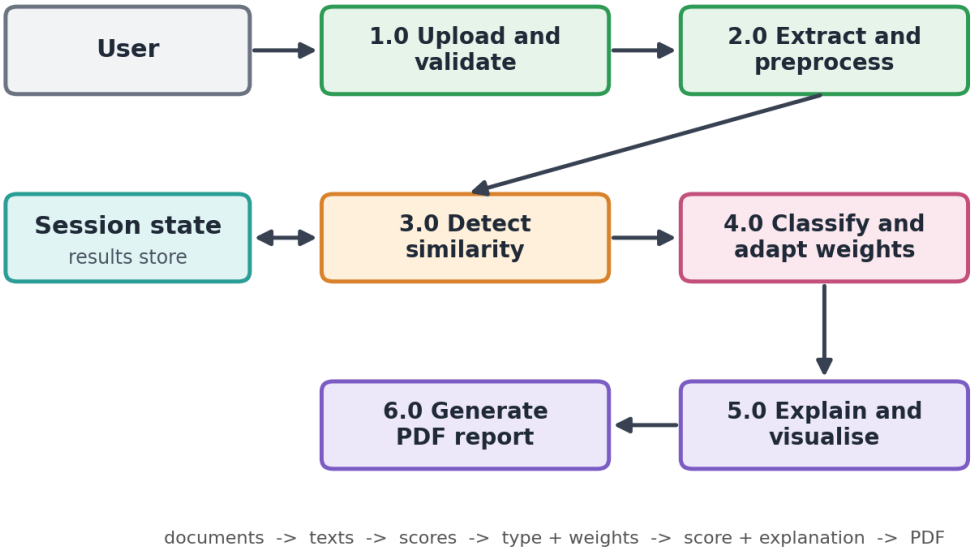


Figure 3.2: Data Flow of the System

Figure 3.3 shows the use cases of the single actor, the user, who may be a student checking their own work or a faculty member screening a submission. All use cases are available from the sidebar menu; the result-related pages require that an analysis has been run first.

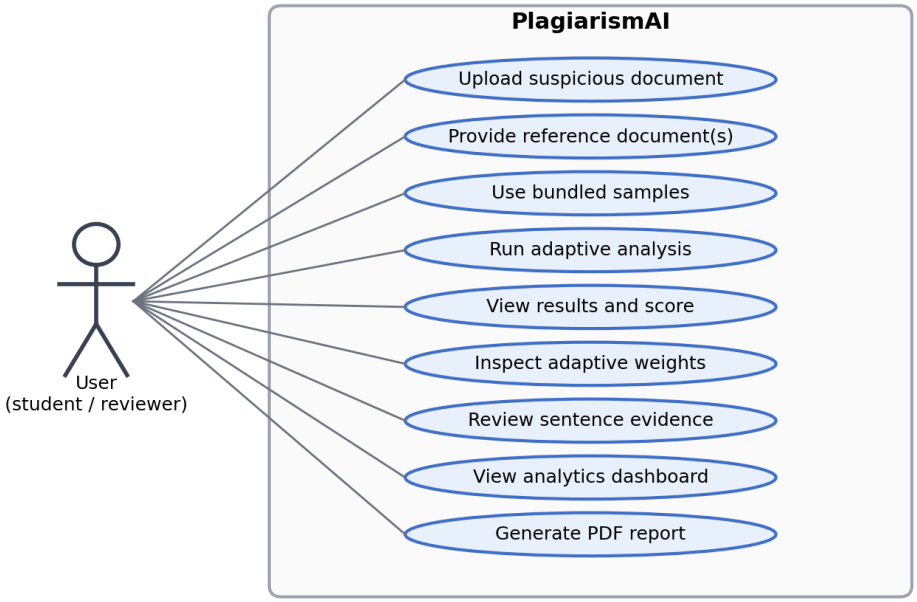


Figure 3.3: Use-Case Diagram

3.6 System Requirements

3.6.1 Functional Requirements

1. The system shall accept one suspicious document and one or more reference documents as PDF, DOCX or TXT files, or let the user choose bundled samples.
2. The system shall extract text from the supplied files and reject files that are empty or unreadable with an explanatory message.
3. The system shall preprocess the text by cleaning, sentence splitting, tokenization, stopword removal and stemming.
4. The system shall compute exact-match, TF-IDF, n-gram and semantic similarity scores for the suspicious document against each reference.
5. The system shall classify the plagiarism type from the pattern of scores and report a classification confidence.
6. The system shall apply the weight profile of the detected type to compute the adaptive plagiarism score, and shall also compute the fixed-weight score for comparison.
7. The system shall identify the most similar source, raise an alert when the score reaches 40% and count the suspicious sentences, which are the sentences with similarity of at least 65%.
8. The system shall display the scores, the weights, the explanation, the sentence-level evidence and the analytics charts on separate pages.
9. The system shall generate a downloadable PDF report of the analysis.

3.6.2 Non-Functional Requirements

- **Usability:** a menu of seven pages, no training required, results explained in plain language.
- **Performance:** a comparison of short documents should return a result within a few seconds on a standard laptop once the embedding model is loaded.
- **Reliability:** the same input should always produce the same scores, and invalid input should produce a clear message rather than a crash.
- **Transparency:** every score shown should be traceable to a named technique and, where applicable, to the sentences that produced it.
- **Privacy and security:** documents are processed locally and are not sent to external services; results are not written to a persistent store.
- **Maintainability and extensibility:** detectors, weight profiles and thresholds can be changed without altering other modules.

- **Portability:** the application runs in a web browser on Windows, Linux and macOS with a standard Python installation.

3.6.3 Hardware and Software Requirements

Table 3.3: System Requirements

Category	Requirement
Processor	Dual-core 2 GHz or better
Memory	4 GB RAM minimum; 8 GB recommended when the embedding model is used
Disk space	About 1 GB for Python packages and the embedding model
Operating system	Windows 10/11, Linux or macOS
Programming language	Python 3.9 or later
Libraries	scikit-learn, NumPy, NLTK, Sentence-Transformers, Streamlit, Plotly, ReportLab, python-docx and a PDF text-extraction library
Development tools	Visual Studio Code, Jupyter Notebook, Git
Browser	Safari, Google Chrome, Mozilla Firefox or Microsoft Edge (the screenshots in Chapter 8 were taken in Safari)

3.7 Input and Output Design

3.7.1 Input Design

Table 3.4: Inputs of the System

Input	Source	Constraint
Suspicious document	File upload, or “Use a bundled sample”	One file; PDF, TXT or DOCX; up to 200 MB
Reference document(s)	File upload (one or more), or “Use bundled sample reference”	PDF, TXT or DOCX; up to 200 MB per file
Run command	“Run Adaptive Plagiarism Analysis” button	Both inputs must be provided
Sentence filters	Two drop-down lists on the Sentence Analysis page	Filter by similarity and by plagiarism type
Report command	“Generate PDF Report” button	An analysis must exist

3.7.2 Output Design

Table 3.5: Outputs of the System by Page

Page	Output
Home	Introduction, buttons “Start Analysis” and “Try Sample Documents”, summary cards and a list of capabilities
Upload & Analyze	Two-step input form with a file uploader or sample selector for the suspicious document and the references
Results	Adaptive plagiarism score, detected type, classification confidence, most similar source, suspicious sentence count, alert message, detection-method comparison chart and the four scores

Page	Output
Adaptive Analysis	Similarity-pattern radar chart, type, confidence, fixed and adaptive scores with their difference, selected dynamic weights, adaptive versus fixed-weight chart and a three-point explanation
Sentence Analysis	Expandable list of compared sentences with similarity values and two filters
Analytics	Dashboard of metrics, four charts and a plagiarism-type summary
Report	PDF report with detection results, adaptive analysis, explanation, suspicious sentences and final conclusion

3.8 Design of the Adaptive Scoring Logic

The adaptive logic is the central design element. It is based on the observation that the four detectors are reliable in different situations and that the pattern of their scores reveals which situation applies. Figure 3.4 shows the idea: the four scores form a vector, the classifier maps the vector to a type, and the type selects the weights with which the scores are combined.

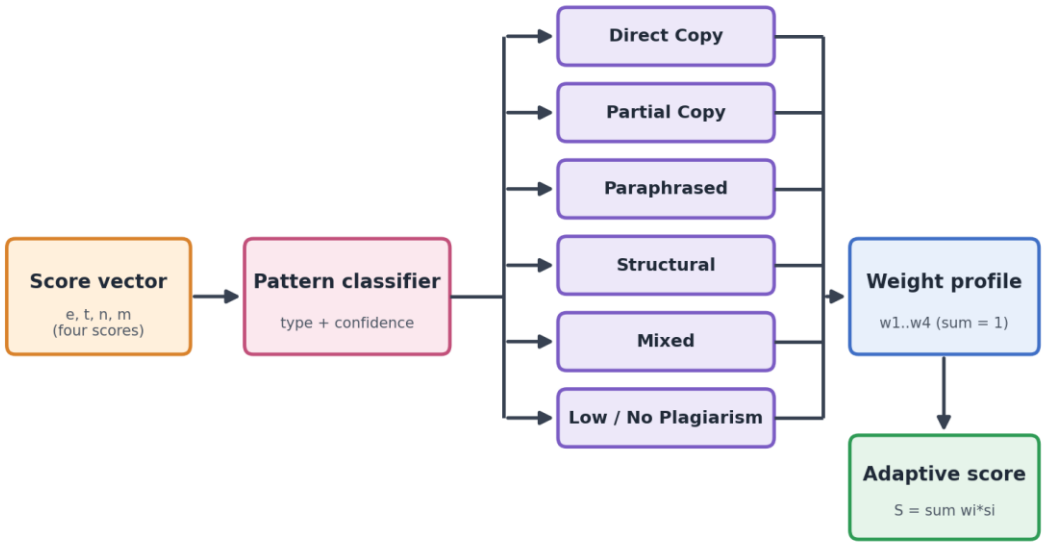


Figure 3.4: Adaptive Type Classification and Weighting

3.8.1 Plagiarism-Type Classification

Let e, t, n and m be the exact-match, TF-IDF, n-gram and semantic scores, each between 0 and 1, and let l = max(e, n) be the strongest lexical signal. The classifier recognises six types. Table 3.6 gives the pattern that characterises each type. The rules are evaluated in the order listed and the first rule that is satisfied decides the type.

Table 3.6: Plagiarism Types and Score Patterns

Type	Characteristic score pattern	Meaning
Direct Copy	$e \geq 0.85$ and $n \geq 0.80$	Almost all text is verbatim

Type	Characteristic score pattern	Meaning
Mixed	$l \geq 0.30$ and $m \geq 0.60$	Lexical and semantic evidence together; some parts copied and others reworded
Partial Copy	$l \geq 0.30$ and $m < 0.60$	A clear part of the text is copied; the rest is original
Structural	$t \geq 0.60$, $m \geq 0.60$ and $l < 0.30$	Same vocabulary and ideas in a different arrangement; exact sequences broken
Paraphrased	$m \geq 0.60$, $t < 0.60$ and $l < 0.30$	Reworded text with preserved meaning
Low / No Plagiarism	None of the above	Only common vocabulary or unrelated text

The classifier also returns a confidence value, which expresses how strongly the score pattern satisfies the rule of the chosen type and is limited to a maximum of 99%. For the sample run in Chapter 8, in which all four scores were 100%, the application reported Direct Copy with a confidence of 99.0%. The thresholds are default design parameters chosen from the behaviour of the detectors on sample passages and are stored as constants so that they can be tuned. They are not values learned from a large dataset, and calibration on a labelled corpus is listed as future work in Chapter 9.

3.8.2 Weight Profiles

Table 3.7 gives the weight profile used for each type, as percentages that sum to 100. A direct copy is judged mostly by exact and lexical evidence, a paraphrase mostly by the semantic detector, and a structural copy by the TF-IDF and semantic detectors, because reordering breaks exact sequences but not shared vocabulary or meaning. For the Direct Copy profile the weights 40, 30, 20 and 10 are displayed on the Adaptive Analysis page. For comparison, the fixed baseline applies the same weights to every document: Exact 20%, TF-IDF 25%, N-gram 20% and Semantic 35%.

Table 3.7: Adaptive Weight Profiles and the Fixed Baseline (%)

Type	Exact	TF-IDF	N-gram	Semantic
Direct Copy	40	30	20	10
Partial Copy	30	20	40	10
Paraphrased	5	20	15	60
Structural	10	35	15	40
Mixed	20	25	25	30
Low / No Plagiarism	20	25	20	35
Fixed baseline (every document)	20	25	20	35

3.8.3 Adaptive Score, Fixed Score and Alert

The adaptive plagiarism score is $S = w_1e + w_2t + w_3n + w_4m$, where the weights are those of the detected type. The traditional fixed score uses the baseline weights, $F = 0.20e + 0.25t + 0.20n + 0.35m$. The difference $S - F$ is shown on the Adaptive Analysis page as the gain of adaptation; a positive value means that adaptation raised the score because the strongest evidence received more weight. If several reference documents are supplied, each is scored separately and the reference with the highest adaptive score is reported as the **most similar source**. The Results page raises an alert, "Plagiarism suspected against <source> (score ≥ 40%)", when that score reaches 40%.

3.8.4 Sentence-Level Evidence

Every sentence of the suspicious document is compared with the sentences of the reference. For each sentence the best-matching reference sentence is kept, together with the combined similarity, the exact-match similarity, the semantic similarity and a label for the match, for example "Exact / Near-Exact Copy". A sentence is called **suspicious** when its similarity is at least 65%. The number of suspicious sentences out of all compared sentences, for example 7 / 7, is shown on the Results page. The Sentence Analysis page lists the sentences, which can be filtered by similarity and by plagiarism type, and the Analytics page plots their distribution against the 65% flag threshold.

3.8.5 Explanation

The explanation is generated from the numbers of the analysis, not from a fixed text. It has three parts: the strongest detector signal and its value; the number of suspicious sentences with the strongest sentence match and its source; and the reason for the weights, namely the detected type, its confidence, the weights applied and why they fit that type. A closing sentence states the conclusion, for example that the document contains substantial verbatim or near-verbatim text matching the reference documents.

> **Design note:** the thresholds of 40% for the alert and 65% for the sentence flag, and the verdict wording, are screening aids. They tell the reviewer where to look first and do not by themselves establish plagiarism.

3.9 Data Handling Design

The documents and the analysis result are held in Streamlit session state while the browser session is open. The result object contains the four scores, the type and confidence, the selected weights, the adaptive and fixed scores, the most similar source, the list of sentence matches and the explanation. The pages Results, Adaptive Analysis, Sentence Analysis, Analytics and

Report read this object and do not recompute the analysis. Nothing is written to a database, which keeps the documents private but also means that the result is lost when the session is closed; the PDF report is therefore the record of an analysis.

3.10 Summary

This chapter described a layered architecture with ten engine modules, the data flow and use cases, the requirements, the input and output design of the seven pages and the adaptive scoring logic with its six plagiarism types, weight profiles, fixed baseline, alert threshold and sentence flag. The next chapter describes how these designs were implemented.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 Introduction

This chapter describes how the design of Chapter 3 was implemented. It explains the implementation approach and then each module in turn, followed by the overall workflow, the use of the application, the handling of data and the security considerations. The corresponding algorithms are given in pseudo-code in Chapter 6, and the resulting application is shown in Chapter 8.

4.2 Implementation Approach

The system is written entirely in Python. The detection engine and the user interface are separate: the engine is a set of functions that take texts and return plain Python data structures, and the Streamlit pages only collect input, call the engine and display the returned result. This separation made it possible to test the detectors in a Jupyter notebook before the interface existed and keeps the report generator independent of the interface. The implementation was carried out in the order of the data flow: document input and extraction, preprocessing, the four detectors, the classifier and weight engine, the explanation and sentence analysis, the charts and finally the PDF report. The source code is available on GitHub (see Appendix B).

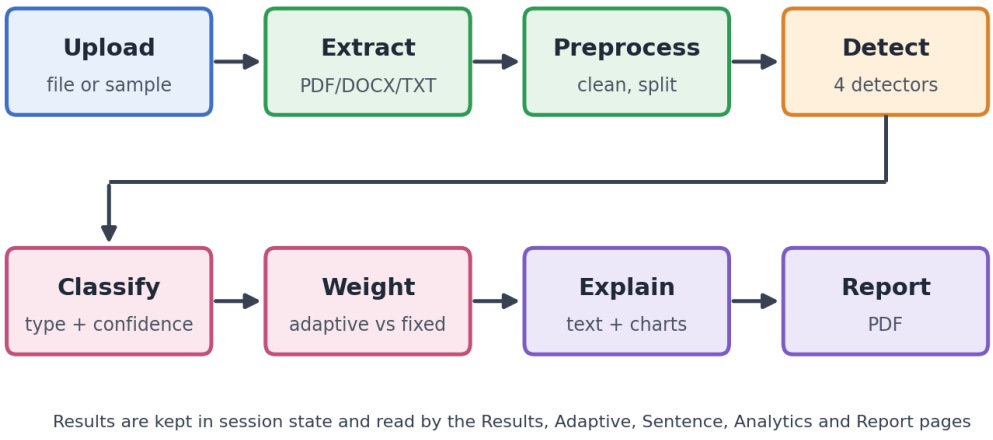


Figure 4.1: Implementation Workflow

4.3 Implementation of the Modules

4.3.1 Document Input and Text Extraction (Modules 1 and 2)

The Upload & Analyze page has two numbered steps. In step 1 the user chooses, with a radio button, either “Upload a file” or “Use a bundled sample” for the suspicious document. In step 2 the user chooses either “Upload file(s)” or “Use bundled sample reference” for the reference

documents, where several files may be selected. The file uploader accepts PDF, TXT and DOCX files of up to 200 MB each and lists the chosen files with their sizes. The “Run Adaptive Plagiarism Analysis” button starts the analysis. TXT files are decoded as text. DOCX files are read paragraph by paragraph. PDF files are read page by page, and their text layers are concatenated. A file from which no text can be read, such as a scanned PDF without a text layer, produces an error message and does not start the analysis.

4.3.2 Preprocessing (Module 3)

Text is first normalised by converting it to lower case, collapsing white space and removing control characters. Sentences are then split at sentence-ending punctuation by a regular expression that also keeps abbreviations and decimal numbers intact, and each sentence is split into word tokens. For the vector-based detectors the tokens are filtered with a stopword list and reduced to stems with the Porter stemmer. The exact-match detector instead uses the lightly normalised word sequence, because stemming and stopword removal would hide verbatim copying and change word order.

4.3.3 Exact Matching (Module 4)

The exact-match detector looks for verbatim word windows of a fixed length k, with k = 8 by default. A window of k consecutive words of the suspicious text that also occurs in the reference marks all of its words as covered. The exact score is the share of the suspicious words that are covered. A document copied word for word is covered completely and scores 100%, while a document with replaced words has few intact windows and scores low. Because the measure uses windows rather than single words, common words such as “the” and “of” do not produce matches by themselves.

4.3.4 TF-IDF Cosine Similarity (Module 5)

The two documents are converted into TF-IDF vectors with the scikit-learn TfidfVectorizer, using the preprocessed text, and the cosine of the angle between the vectors is the TF-IDF score. The measure responds to shared vocabulary and tolerates a change of word order, so it stays high for copied text and for structurally rearranged text, but it does not recognise synonyms.

4.3.5 N-gram Overlap (Module 6)

The n-gram detector builds the set of stemmed word trigrams of the suspicious text and measures the fraction that also occurs in the reference. This is the containment measure of Broder [12]. It is high for copied and lightly edited text, falls quickly when words are replaced,

and, because it divides by the size of the suspicious text, it correctly gives a high score when a short submission is wholly contained in a long source.

4.3.6 Semantic Similarity with Sentence-BERT (Module 7)

The semantic detector encodes every sentence of both documents with the Sentence-BERT model all-MiniLM-L6-v2 [7], [17] into 384-dimensional vectors. For each sentence of the suspicious document it finds the reference sentence with the highest cosine similarity, and the semantic score is the mean of these best-match similarities. The model is loaded once and kept in memory, so only the first analysis of a session pays the loading time. The same sentence vectors are reused by the sentence analysis, so the sentences are not encoded twice.

4.3.7 Plagiarism-Type Classification (Module 8)

The classifier receives the four scores and evaluates the rules of Table 3.6 in order. It returns the type name and the confidence, which is limited to 99%. The result is also used by the radar chart on the Adaptive Analysis page: because the chart plots the same four scores on its axes, its shape makes the pattern visible. A diamond that fills all four axes indicates a copy; a shape that is large on the semantic axis and small on the others indicates a paraphrase.

4.3.8 Adaptive Weighting and Fixed-Weight Comparison (Modules 9 and 10)

The weight engine looks up the profile of the detected type in a table of constants (Table 3.7) and computes the adaptive score as the weighted sum of the four scores. Module 10 computes the same sum with the fixed baseline weights (Exact 20%, TF-IDF 25%, N-gram 20%, Semantic 35%) and subtracts it from the adaptive score. The page shows the traditional fixed score, the adaptive score and the difference as a metric with an arrow, and draws both scores side by side in the “Adaptive vs Fixed-Weight Scoring” chart. A caption under the chart states the fixed baseline weights so that the comparison can be checked.

4.3.9 Explanation Generator

The explanation is composed from templates that are filled with the numbers of the current result. The first sentence names the strongest detector and its score. The second sentence reports how many of the compared sentences were flagged, the strongest sentence match, the source file and the label of that match. The third sentence explains the weights: it names the detected pattern and its confidence, lists the four weights applied and gives the reason, for example that exact and lexical matching dominate because the content was copied near-verbatim. A closing statement summarises the conclusion and the score. The same text is shown on the Adaptive Analysis page and printed in the PDF report.

4.3.10 Sentence-Level Analysis

The sentence analysis uses the sentence embeddings and the sentence tokens to find, for every suspicious sentence, the best-matching reference sentence. For each pair it records the combined similarity, the exact similarity, the semantic similarity and a label. Pairs with a similarity of at least 65% are marked as suspicious. On the Sentence Analysis page each pair is an expandable row headed by a coloured marker, the similarity percentage and the beginning of the sentence; the two drop-down lists filter the rows by similarity and by plagiarism type, and a caption shows how many of the compared sentences are displayed.

4.3.11 Analytics Dashboard

The Analytics page repeats the five key figures and then draws four charts with Plotly: the detection-method comparison, the adaptive weight selection, the similarity-pattern radar and the sentence-level similarity distribution, in which a dashed line marks the 65% flag threshold. A closing section, “Plagiarism Type Summary”, states the detected type and its confidence. Placing the charts side by side allows the reviewer to relate the scores to the weights chosen and to the distribution of the sentence similarities.

4.3.12 PDF Report Generation

The Report page offers a “Generate PDF Report” button. When it is pressed the result object is passed to a report builder that uses ReportLab to create a document with the title “AI Based Plagiarism Detection System: Adaptive Plagiarism Analysis Report”, the analysed document, the reference documents and the analysis date and time, followed by the sections Detection Results, Adaptive Analysis, Why Was This Flagged? (Explainable AI), Suspicious Sentences and Final Conclusion. Each suspicious sentence is printed with its matched source sentence, similarity, exact and semantic values and the type label. The PDF can then be downloaded.

4.4 Workflow of the Application

The normal workflow has six steps. (1) The user opens the Home page and presses “Start Analysis”, or “Try Sample Documents” to load the bundled samples. (2) On Upload & Analyze the user provides the suspicious document and the references and presses “Run Adaptive Plagiarism Analysis”. (3) The engine extracts and preprocesses the texts, runs the four detectors for each reference, classifies the type, applies the adaptive weights and prepares the sentence matches and the explanation. (4) The user opens the Results page for the headline figures and the alert, then Adaptive Analysis for the pattern, the weights and the explanation. (5) Sentence Analysis and Analytics give the evidence in detail. (6) The Report page produces the PDF.

4.5 Use Cases

Table 4.1: Use Cases of the System

Use case	Actor and situation	What the system provides
Self-check before submission	A student compares a draft with the sources used	Score, type, flagged sentences and the PDF to correct the draft
Screening a submission	A faculty member compares a report with earlier submissions or a known source	Most similar source, alert, sentence evidence and the report for the file
Resume or portfolio check	A reviewer compares a resume or project description with published text	Short-text comparison with type classification
Demonstration	A user tries the bundled samples from the Home page	A complete analysis without preparing files

4.6 Module Interaction

Figure 4.2 shows the call structure of the engine. The application pages call three function groups. The loader reads the text of a TXT, DOCX or PDF file and rejects a file in which no readable text is found. The analysis function runs, in order, the preprocessing, the four detectors, the type classifier, the adaptive weighting, the sentence analysis and the explanation generator, and returns one result object to the page. The report generator builds the tables and the sentence evidence from that result object and creates the PDF with ReportLab.

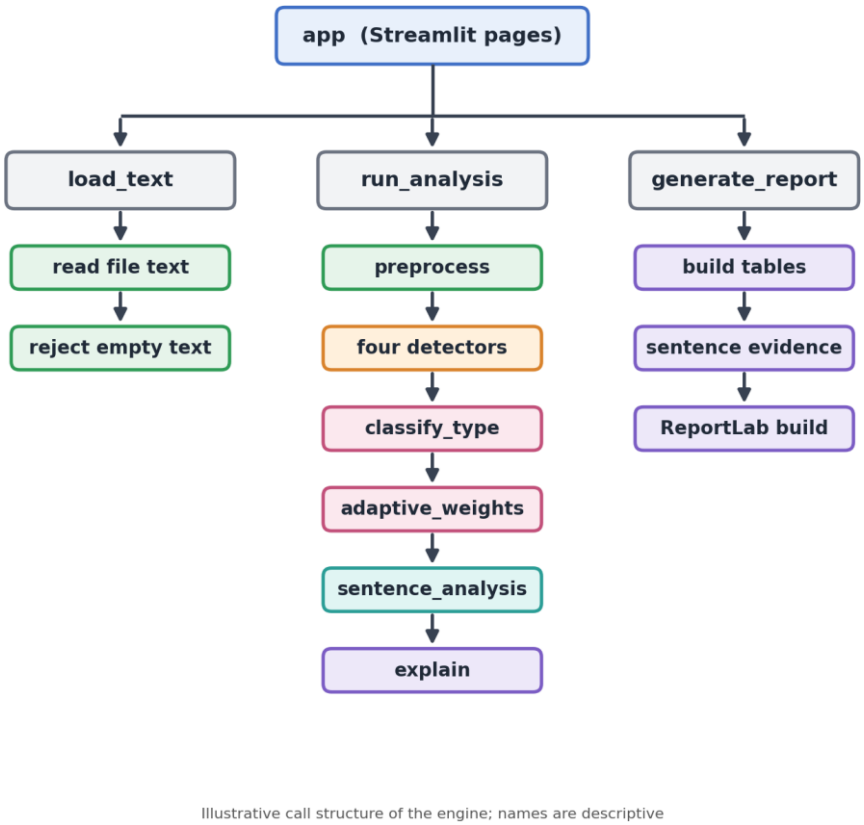


Figure 4.2: Interaction of the Engine Functions

4.7 Data Handling and Security

- **Local processing.** Uploaded documents are processed in the memory of the machine that runs the application. The only network access needed is the one-time download of the embedding model.
- **No persistent storage.** Documents and results are kept in session state and disappear when the session ends. The PDF report is the only artefact that leaves the application, and only when the user requests it.
- **Input validation.** The upload widget accepts only PDF, TXT and DOCX files within the 200 MB limit, and unreadable or empty files are rejected before the analysis starts.
- **Access control.** The prototype has no login. It is intended for a single user on a local machine; authentication is listed as future scope.
- **Decision support.** The application states that scores are screening aids. Results should be verified by reading the matched sentences.

4.8 Summary

The implementation follows the layered design: a separate engine of ten modules, a seven-page Streamlit interface that reads one shared result, interactive Plotly charts and a ReportLab report. The next chapters describe the technologies used, the algorithms and the testing.

CHAPTER 5

TECHNOLOGIES USED

5.1 Introduction

This chapter describes the main technologies used to build the system and explains why each was chosen. The selection was guided by four criteria: the technology should be free and open source, widely used and well documented, suitable for text processing and machine learning, and light enough to run on a student laptop.

5.2 Python

Python is the programming language of the whole project. It is the dominant language for NLP and machine learning because of its readable syntax and its large ecosystem of libraries for text processing, numerical computing, charting and web applications. A single language for the document parsing, the preprocessing, the models, the interface and the report generation avoids the integration work that a multi-language system would need. Python's standard library also supplies the regular-expression module used for tokenization and sentence splitting.

5.3 NLTK and Regular Expressions

The Natural Language Toolkit (NLTK) is a Python library for working with human language data [16]. The project uses its Porter stemmer, which applies a sequence of suffix-stripping rules to reduce words such as "detection", "detects" and "detecting" to a common stem. The Porter stemmer needs no external data files, so it can be used without downloading corpora. Tokenization and sentence splitting are implemented with Python regular expressions, which gives complete control over what counts as a token and avoids the need for additional language models.

spaCy [19], an industrial-strength NLP library, was considered for tokenization and lemmatization. The final implementation uses lightweight regular expressions and stemming because they are fast, deterministic and sufficient for the vector-based detectors. spaCy remains an option for lemmatization and for the multilingual extension discussed in Chapter 9.

5.4 scikit-learn

scikit-learn is a Python library of machine-learning algorithms with a consistent interface [14]. The project uses three parts of it. The **TfidfVectorizer** converts text into sparse matrices of TF-IDF weights and supports word n-grams. The **cosine_similarity** function computes the cosine of the angle between vectors. The **ENGLISH_STOP_WORDS** list provides the

stopwords. TF-IDF multiplies the frequency of a term in a document by a factor that decreases with the number of documents in which the term occurs, so terms that characterise a document receive high weights and terms that occur everywhere receive low ones [6]. Cosine similarity of two weighted vectors is close to 1 when they share the same important terms and close to 0 when they share none.

5.5 Sentence-Transformers and Sentence-BERT

BERT [8] is a Transformer-based language model [15] that is pre-trained on large text collections. Used directly, it is expensive for similarity search because every pair of sentences must be processed together. Sentence-BERT [7] solves this by using a siamese network structure that produces a fixed-size vector for each sentence independently, so vectors can be computed once and compared by cosine similarity. The Sentence-Transformers library provides ready-to-use models of this kind.

The default model in this project is all-MiniLM-L6-v2, which is built on the compact MiniLM architecture [17]. It has six Transformer layers and produces 384-dimensional sentence vectors. It was chosen because it is small enough to run on a CPU in a short time while still giving good sentence-level semantic similarity. The model name is a parameter of the semantic detector, so a larger or multilingual model can be substituted without changing the rest of the code.

5.6 NumPy

NumPy provides the array operations on which scikit-learn and Sentence-Transformers are built. In this project it is used to hold the weight vectors and score vectors, to compute the weighted sum, to multiply the embedding matrices of the two documents and to take the row-wise maximum when finding the best-matching sentence for each submitted sentence.

5.7 Streamlit

Streamlit [20] is a Python framework for building interactive web applications from ordinary scripts. Interface elements such as text areas, file uploaders, buttons, tables and charts are created by calling functions, and the page is re-run whenever the user interacts with it. This allowed the interface to be built quickly and to remain in the same language and process as the engine. Session state is used to remember the most recent result across interactions. Streamlit applications run in the browser, so no installation is needed on the user’s side beyond a web browser.

5.8 Plotly and Document Parsers

Plotly [22] is an open-source graphing library whose Python interface produces interactive charts that Streamlit can embed directly. The project uses it for the detection-method comparison bar chart, the similarity-pattern radar chart, the adaptive-weight bar chart, the adaptive versus fixed-weight comparison and the sentence-similarity histogram with its flag threshold. The charts offer zoom, pan and image export, which helps a reviewer inspect the numbers. Uploaded files are converted to plain text before analysis: TXT files are decoded directly, DOCX files are read paragraph by paragraph with the python-docx library, and PDF files are read page by page with a PDF text-extraction library. Scanned PDFs that contain only images carry no text layer and are therefore not supported.

5.9 ReportLab

ReportLab [21] is a Python library for generating PDF documents programmatically. Its Platypus layout engine arranges paragraphs, tables and spacers into pages. The project uses it to create the downloadable report. The report is built from the result object and does not depend on the interface, so it can also be generated from a script.

5.10 Development Environment

Development was carried out in Visual Studio Code, which provides code editing, a debugger and Git integration, and in Jupyter Notebook, which was used to experiment with the detectors and to inspect intermediate values such as TF-IDF matrices and similarity matrices before the code was moved into the engine module. Version control was handled with Git. The interface was tested in Google Chrome, Mozilla Firefox and Microsoft Edge to confirm consistent behaviour. The source code is hosted on GitHub [23] (see Appendix B).

5.11 Mathematical Foundations

The detectors rely on a small number of formulas, which are collected here so that the numbers in the following chapters can be checked.

Table 5.1: Formulas Used by the Detectors

Measure	Formula	Notation
TF-IDF weight [6], [14]	$w(t, d) = tf(t, d) \times idf(t)$, $idf(t) = \ln((1 + N) / (1 + df(t))) + 1$	tf: count of term t in document d; N: number of documents; df: documents containing t
Cosine similarity	$\cos(a, b) = (a \cdot b) / (\|a\| \times \|b\|)$	a, b: weighted term vectors (or embeddings)
N-gram containment [12]	$C(A, B) = \|G(A) \cap G(B)\| / \|G(A)\|$	G(X): set of word n-grams of text X
Exact-match coverage	$E(A, B) = \|\{$words of A inside a verbatim k-window found in B$\}\| / \|A\|$	k = 8 by default
Semantic score [7]	$M(A, B) = (1/s) \Sigma_i \max_j \cos(u_i, v_j)$	u_i: embedding of the i-th sentence of A; v_j: embedding of the j-th sentence of B; s: sentences in A
Final score	$S = w_1E + w_2T + w_3C + w_4M$, $\Sigma w = 1$	T: TF-IDF score; C: n-gram score; weights from the profile of the detected type

The inverse document frequency in the table is the form used by scikit-learn with its default smoothing [14]. Because the vectoriser is fitted on two documents only, a term that occurs in both documents has df = 2 and idf = ln(3/3) + 1 = 1, whereas a term that occurs in only one has df = 1 and idf = ln(3/2) + 1 ≈ 1.41, so terms found in only one of the documents receive a somewhat larger weight. Vectors are normalised to unit length by default, so the cosine similarity equals the dot product.

Why containment rather than the Jaccard index. The Jaccard index $|G(A) \cap G(B)| / |G(A) \cup G(B)|$ is large only when the two sets are of similar size. If a student copies one paragraph from a long article, the union is dominated by the article, and the Jaccard index is small even though the whole submission is copied. Containment divides by $|G(A)|$ instead, so it equals 1 when everything in the submission appears in the source [12].

5.12 Alternatives Considered

Table 5.2 lists the alternatives that were considered for the main components and the reason for the choice made.

Table 5.2: Technology Choices and Alternatives

Component	Chosen	Alternatives considered	Reason for the choice
Semantic model	all-MiniLM-L6-v2 [17]	Larger Sentence-BERT models; word-embedding averages [9], [10]	Small and fast on a CPU; gives sentence-level vectors that word-embedding averages cannot
Lexical comparison	Sets of hashed n-grams	Winnowing fingerprints [11]	Sets are simple and exact for pairwise comparison; fingerprints become useful for large collections
Stemming	Porter stemmer	Lemmatization (spaCy [19])	Stemming needs no model download and is fast; lemmatization is a possible future refinement
Interface	Streamlit	Flask or Django with HTML; React with a REST API	The interface stays in Python and in the same process as the engine, which shortens development
Result handling	Streamlit session state	A database (SQLite, MySQL, PostgreSQL); JSON files	Streamlit session state is sufficient because the latest result is the only state the pages share; a database is future scope
Reports	ReportLab	HTML-to-PDF converters	Direct control of the PDF from Python without extra programs
Combining scores	Type-dependent fixed weights	Single average; learned classifier	Transparent and needs no training data; a learned combiner is future work

5.13 Summary of Technologies Used

Table 5.3: Summary of Technologies

Technology	Category	Role in the system
Python	Language	Implementation of all modules
NLTK	NLP library	Porter stemming
Regular expressions	Standard library	Tokenization and sentence splitting
scikit-learn	ML library	TF-IDF, cosine similarity, stopword list
Sentence-Transformers	NLP / deep-learning library	Sentence-BERT embeddings (all-MiniLM-L6-v2)
NumPy	Numerical library	Weighted scoring and matrix operations
Streamlit	Web framework	User interface
Plotly	Charting library	Interactive charts (bar, radar, histogram)
ReportLab	PDF library	Report generation
VS Code, Jupyter, Git	Tools	Development, experiments and version control

Together these technologies make up a complete and entirely open-source stack. Each is replaceable: for example, the semantic detector could use a different model, results could be stored in a database, and the interface could be replaced by a REST API without changing the other modules.

CHAPTER 6

IMPLEMENTATION

6.1 Introduction

This chapter gives the main algorithms of the system. They are written as pseudo-code that follows the structure of the implementation described in Chapter 4, so that the logic can be read without the details of the Python libraries. The chapter first lists the structure of the application pages and then gives the algorithms in the order in which they run.

6.2 Structure of the Application

The application is a single Streamlit program. A radio-button menu in the sidebar selects the page to be drawn, and the page functions read and write the shared session state. Table 6.1 lists the pages, what each one reads and the main function it uses.

Table 6.1: Application Pages and Their Roles

Page	Reads from session	Main action	Charts / widgets
Home	Nothing	Introduce the system; buttons to start or use samples	Hero banner, summary cards
Upload & Analyze	Nothing	Collect inputs; run the engine and store the result	Radio buttons, file uploaders, run button
Results	Result object	Show headline figures, alert and detector scores	Bar chart, metrics
Adaptive Analysis	Result object	Show pattern, weights, fixed versus adaptive score, explanation	Radar, two bar charts
Sentence Analysis	Sentence matches	List and filter sentence pairs	Expanders, two filters
Analytics	Result object	Dashboard of all charts and the type summary	Four charts
Report	Result object	Build and offer the PDF	Generate button, download

6.3 Algorithms

Algorithm 6.1: Document Loading and Text Extraction

```
FUNCTION load_text(file):
    extension = lower-case file extension
    IF extension = "txt":   text = decode(file.bytes)
    ELSE IF extension = "docx": text = join(paragraph.text FOR paragraph IN file)
    ELSE IF extension = "pdf":  text = join(page.text FOR page IN file.pages)
    ELSE: RAISE unsupported file type
    IF length(trim(text)) = 0: RAISE no readable text
    RETURN text
```

Algorithm 6.2: Preprocessing

```
FUNCTION preprocess(text):
    clean     = collapse_whitespace(lower(text))
    sentences = split_sentences(clean)
    words     = tokenize(clean)                      # used by exact matching
    stems     = [stem(w) FOR w IN words IF w NOT IN stopwords]
    RETURN clean, sentences, words, stems
```

Algorithm 6.3: Exact Matching and N-gram Containment

```
FUNCTION exact_score(words_A, words_B, k = 8):
    windows_B = SET(tuple(words_B[i : i+k]) FOR i IN range(len(words_B)-k+1))
    covered   = boolean array of length len(words_A), all False
    FOR i IN range(len(words_A)-k+1):
        IF tuple(words_A[i : i+k]) IN windows_B: covered[i : i+k] = True
    RETURN count(covered) / len(words_A)

FUNCTION ngram_score(stems_A, stems_B, n = 3):
    G_A = SET(n-grams of stems_A);  G_B = SET(n-grams of stems_B)
    RETURN |G_A INTERSECT G_B| / |G_A|
```

Algorithm 6.4: TF-IDF and Semantic Similarity

```
FUNCTION tfidf_score(text_A, text_B):
    vectors = TfidfVectorizer().fit_transform([text_A, text_B])
    RETURN cosine_similarity(vectors[0], vectors[1])

FUNCTION semantic_score(sentences_A, sentences_B):
    U = encode(sentences_A);  V = encode(sentences_B)       # Sentence-BERT
    S = cosine_matrix(U, V)                                 # |A| x |B|
    RETURN mean(max(S[i, :]) FOR i IN range(|A|))
```

Algorithm 6.5: Plagiarism-Type Classification

```
FUNCTION classify(e, t, n, m):
    l = max(e, n)
    IF e >= 0.85 AND n >= 0.80:              type = "Direct Copy"
    ELSE IF l >= 0.30 AND m >= 0.60:         type = "Mixed"
    ELSE IF l >= 0.30:                       type = "Partial Copy"
    ELSE IF t >= 0.60 AND m >= 0.60:         type = "Structural"
    ELSE IF m >= 0.60:                       type = "Paraphrased"
    ELSE:                                    type = "Low / No Plagiarism"
    confidence = min(0.99, strength_of_match(type, e, t, n, m))
    RETURN type, confidence
```

Algorithm 6.6: Adaptive and Fixed-Weight Scoring

```
PROFILE = {"Direct Copy": (40,30,20,10), "Partial Copy": (30,20,40,10),
           "Paraphrased": (5,20,15,60), "Structural": (10,35,15,40),
           "Mixed": (20,25,25,30), "Low / No Plagiarism": (20,25,20,35)}
FIXED   = (20, 25, 20, 35)

FUNCTION score(type, e, t, n, m):
    w        = PROFILE[type] / 100
    adaptive = w[0]*e + w[1]*t + w[2]*n + w[3]*m
    fixed    = 0.20*e + 0.25*t + 0.20*n + 0.35*m
    RETURN adaptive, fixed, adaptive - fixed
```

Algorithm 6.7: Sentence-Level Analysis

```
FUNCTION sentence_matches(sent_A, sent_B, emb_A, emb_B, flag = 0.65):
    matches = []
    FOR i IN range(len(sent_A)):
        j        = argmax(cosine(emb_A[i], emb_B[j]) FOR j)
        exact    = lexical_similarity(sent_A[i], sent_B[j])
        semantic = cosine(emb_A[i], emb_B[j])
        similarity = combine(exact, semantic)
        label    = match_label(exact, semantic)    # e.g. "Exact / Near-Exact Copy"
        matches.append(sent_A[i], sent_B[j], similarity, exact, semantic, label,
                       suspicious = similarity >= flag)
    RETURN matches
```

Algorithm 6.8: Complete Analysis

```
FUNCTION run_analysis(suspicious_file, reference_files):
    A = preprocess(load_text(suspicious_file))
    results = []
    FOR EACH reference IN reference_files:
        B = preprocess(load_text(reference))
        e = exact_score(A.words, B.words);   t = tfidf_score(A.clean, B.clean)
        n = ngram_score(A.stems, B.stems);   m = semantic_score(A.sentences,
B.sentences)
        type, conf = classify(e, t, n, m)
        adaptive, fixed, gain = score(type, e, t, n, m)
        results.append(reference, e, t, n, m, type, conf, adaptive, fixed, gain)
    best = result with the highest adaptive score          # most similar source
    best.sentences   = sentence_matches(A.sentences, best.sentences, ...)
    best.explanation = explain(best)
    best.alert       = best.adaptive >= 0.40
    session_state["result"] = best
    RETURN best
```

Algorithm 6.9: Explanation and Report

```
FUNCTION explain(r):
    s1 = "<strongest detector> similarity was the strongest signal at <value>."
    s2 = "<k> of <N> compared sentences were flagged ... strongest match <x>% with
<source>."
    s3 = "Because the pattern matched <type> (confidence <c>), the adaptive engine
          weighted the detectors as: Exact <w1>, TF-IDF <w2>, N-gram <w3>, Semantic
<w4>."
    RETURN [s1, s2, s3], conclusion(r.type, r.adaptive)

FUNCTION build_pdf(r):
    story = [title, metadata table, detection results table, adaptive analysis table,
             explanation, suspicious sentences with matched sources, final
conclusion]
    RETURN ReportLab.build(story)
```

6.4 Page Control Flow

Each page function begins by checking whether it needs a result. The Upload & Analyze page calls Algorithm 6.8 when the run button is pressed and stores the result in session state. The pages from Results to Report test whether a result exists; if it does not, they show a message asking the user to run an analysis first, and otherwise they read the stored object and draw it. The Report page calls Algorithm 6.9 only when the “Generate PDF Report” button is pressed,

so no PDF is built unless it is wanted. Because the pages share one result, the numbers on the Results, Adaptive Analysis and Analytics pages and in the PDF always agree.

6.5 Complexity and Performance

The exact and n-gram detectors run in time proportional to the length of the two documents because they use hash sets. The TF-IDF detector is linear in the vocabulary. The semantic detector dominates the cost: encoding takes time proportional to the number of sentences, and the best-match search compares every suspicious sentence with every reference sentence, which is quadratic in the number of sentences. For documents of a few pages this takes seconds; for very large collections an index of precomputed embeddings would be needed, as noted in Chapter 9.

6.6 Summary

The chapter listed the pages of the application and gave nine algorithms: loading, preprocessing, the four detectors, classification, adaptive and fixed scoring, sentence analysis, the complete analysis and the explanation and report. The next chapter describes how the system was tested.

CHAPTER 7

TESTING

7.1 Introduction

Testing checks that the system behaves as designed and shows where it does not. This chapter describes the test strategy and environment, lists the test cases for each page of the application and the engine, and reports the result that has been observed so far, a run in which a document was compared with itself. It also explains what that run proves and what it cannot prove, and lists the further tests that are needed to evaluate the adaptive behaviour on reworded text.

7.2 Test Strategy

- **Unit testing** of the engine functions: preprocessing, each detector, the classifier, the weight lookup and the score calculation, using short texts whose result can be worked out by hand.
- **Functional testing** of the seven pages through the browser, following the use cases of Chapter 4.
- **Boundary and error testing** with empty, unreadable, very short and unsupported files.
- **Consistency testing**: the figures on the Results, Adaptive Analysis and Analytics pages and in the PDF report must agree.
- **Behavioural testing** with sample pairs of increasing rewording, to see whether the type and the weights change as designed.

7.3 Test Environment

Table 7.1: Test Environment

Item	Description
Application	PlagiarismAI v1.0, run locally with Streamlit and opened at localhost
Browser	Safari
Input files	doc1.txt (1.1 KB, 7 sentences, on the use of artificial intelligence in education) as suspicious document and as reference
Detectors	Exact match, TF-IDF with cosine similarity, n-gram overlap, Sentence-BERT (all-MiniLM-L6-v2)
Date of recorded run	29 October 2026 (shown in the PDF report as 2026-10-29 18:49)

7.4 Test Cases

Table 7.2 lists the test cases. The column “Result” states the outcome of the cases that have been executed and recorded with a screenshot; the other cases are defined and are to be executed on the final build.

Table 7.2: Test Cases

ID	Test	Input / action	Expected result	Result
T1	Home page loads	Open the application	Banner, two buttons, four summary cards and the capability list are shown	Passed
T2	Upload a suspicious file	Choose “Upload a file” and select doc1.txt	File is listed with name and size (1.1 KB)	Passed
T3	Upload a reference file	Choose “Upload file(s)” and select doc1.txt	File is listed with name and size	Passed
T4	Run analysis on identical documents	Press “Run Adaptive Plagiarism Analysis”	All four scores 100%; type Direct Copy; alert shown	Passed
T5	Results page figures	Open Results	Score 100.0%, type Direct Copy, confidence 99.0%, source doc1.txt, 7 / 7 sentences	Passed
T6	Adaptive weights	Open Adaptive Analysis	Weights 40 / 30 / 20 / 10; fixed score 100.0%; difference +0.0	Passed
T7	Sentence list	Open Sentence Analysis	“Showing 7 of 7 compared sentences”, each at 100.0%	Passed
T8	Analytics dashboard	Open Analytics	Four charts and “Direct Copy — confidence 99.0%”	Passed
T9	PDF report	Press “Generate PDF Report”	Two-page PDF with all sections and 7 suspicious sentences	Passed
T10	Unsupported file type	Upload a .png file	File is not accepted by the uploader	To be executed
T11	Empty or unreadable file	Upload an empty .txt or a scanned PDF	Error message; analysis does not start	To be executed
T12	Several references	Upload two or more references	Highest-scoring reference reported as most similar source	To be executed
T13	Lightly edited copy	Edit a few words of doc1.txt	Partial Copy or Direct Copy; lexical scores below 100%	To be executed
T14	Paraphrased text	Reword every sentence of doc1.txt	Type Paraphrased; semantic score highest; weights favour semantic	To be executed
T15	Unrelated text	Compare with a text on another subject	Type Low / No Plagiarism; no alert	To be executed
T16	Bundled samples	Use “Try Sample Documents”	Analysis runs without uploading files	To be executed

7.5 Observed Result: Identical Documents

In the recorded run, doc1.txt was supplied both as the suspicious document and as the reference. Because the two texts are identical, the correct behaviour is that every detector reports complete similarity and every sentence is flagged. Table 7.3 shows the observed values.

Table 7.3: Observed Values for the Identical-Document Run

Quantity	Observed	Expected
Exact match score	100.0%	100%
TF-IDF score	100.0%	100%
N-gram score	100.0%	100%
Semantic (Sentence-BERT) score	100.0%	100%

Quantity	Observed	Expected
Detected type / confidence	Direct Copy / 99.0%	Direct Copy
Weights applied (Exact / TF-IDF / N-gram / Semantic)	40% / 30% / 20% / 10%	Direct Copy profile
Adaptive score	100.0%	100%
Traditional fixed score	100.0%	100%
Difference (adaptive − fixed)	+0.0 points	0 for equal scores
Suspicious sentences	7 / 7	7 / 7
Alert	Plagiarism suspected against doc1.txt (score ≥ 40%)	Alert

7.6 Verification of the Score Calculation

The arithmetic of the scoring can be checked by hand. With e = t = n = m = 1, the adaptive score is 0.40 + 0.30 + 0.20 + 0.10 = 1.00 and the fixed score is 0.20 + 0.25 + 0.20 + 0.35 = 1.00, both 100%, which agrees with the Results and Adaptive Analysis pages. The same check applies to every profile in Table 3.7: because each profile sums to 100%, identical scores on all four detectors always give the same adaptive and fixed result, whatever the type.

This observation is also a limitation of the recorded run. A document compared with itself cannot show the benefit of adaptive weighting, because adaptive and fixed scores coincide when all four scores are equal. The benefit appears only when the detector scores differ, for example when lexical scores are low and the semantic score is high. Table 7.4 predicts the behaviour for such cases from the design, so that the planned tests T13 to T15 have a stated expectation.

Table 7.4: Expected Behaviour for Planned Behavioural Tests

Test pair	Expected score pattern	Expected type	Weights favour
Identical text	All four scores near 100%	Direct Copy	Exact and TF-IDF
A few words changed	Exact falls, TF-IDF and semantic remain high	Direct Copy or Partial Copy	Exact and N-gram
Half of the sentences copied	Moderate lexical, moderate semantic	Partial Copy or Mixed	N-gram and Exact
Fully reworded	Exact and N-gram low; semantic high	Paraphrased	Semantic
Same sentences reordered	Exact low; TF-IDF and semantic high	Structural	TF-IDF and Semantic
Unrelated subject	All four scores low	Low / No Plagiarism	Baseline

7.7 Observations and Limitations of the Testing

1. The recorded run confirms that the full pipeline from upload to PDF report works end to end and that the pages agree with each other.

2. The values of the type thresholds and of the profiles other than Direct Copy have not yet been checked by running reworded text, so the behaviour for paraphrased, structural and mixed cases is specified but not yet measured.
3. No accuracy, precision or recall has been measured, because no labelled collection of document pairs has been used.
4. The execution time of an analysis was not recorded in this run.
5. In the Sentence Analysis page the title of each expandable row shows only the beginning of the sentence; the full sentence and its matched source are inside the expanded row and in the PDF report.

7.8 Summary

The system passed the nine test cases that have been executed, which cover the whole workflow from upload to PDF for an identical-document run. Seven further tests, covering error handling, several references and the behavioural cases, are defined for the final build. The results of an evaluation on reworded text are the most important missing evidence and are listed first in the future scope.

CHAPTER 8

SAMPLE OUTPUT

8.1 Introduction

This chapter shows the output of the system for the recorded run. It first describes the sample document and then shows each page of the application in the order in which a user meets it, followed by the generated PDF report. The values quoted are those displayed by the application.

8.2 Sample Document

The sample document doc1.txt has a size of 1.1 KB and contains seven sentences on the use of artificial intelligence in education. Its sentences describe how education systems can personalise learning, how a system can analyse a student's performance and recommend learning material, and how teachers can use the information to monitor progress. The same file was supplied as the suspicious document and as the reference, so the expected result is complete similarity.

8.3 Home Page

The Home page (Figure 8.1) introduces the system under the heading "AI Based Plagiarism Detection System" and the tag "Adaptive · Explainable · AI-Powered". Two buttons, "Start Analysis" and "Try Sample Documents", lead into the application. Four cards summarise the system: 4 detection techniques, 6 plagiarism types classified, PDF · DOCX · TXT as supported formats, and multi-source comparison against many references. Below them, the "What it can do" section lists six capabilities: multi-technique detection, plagiarism type classification, the adaptive weighting engine, explainable results, adaptive versus fixed comparison and sentence-level evidence. The sidebar on the left holds the navigation menu with the seven pages.

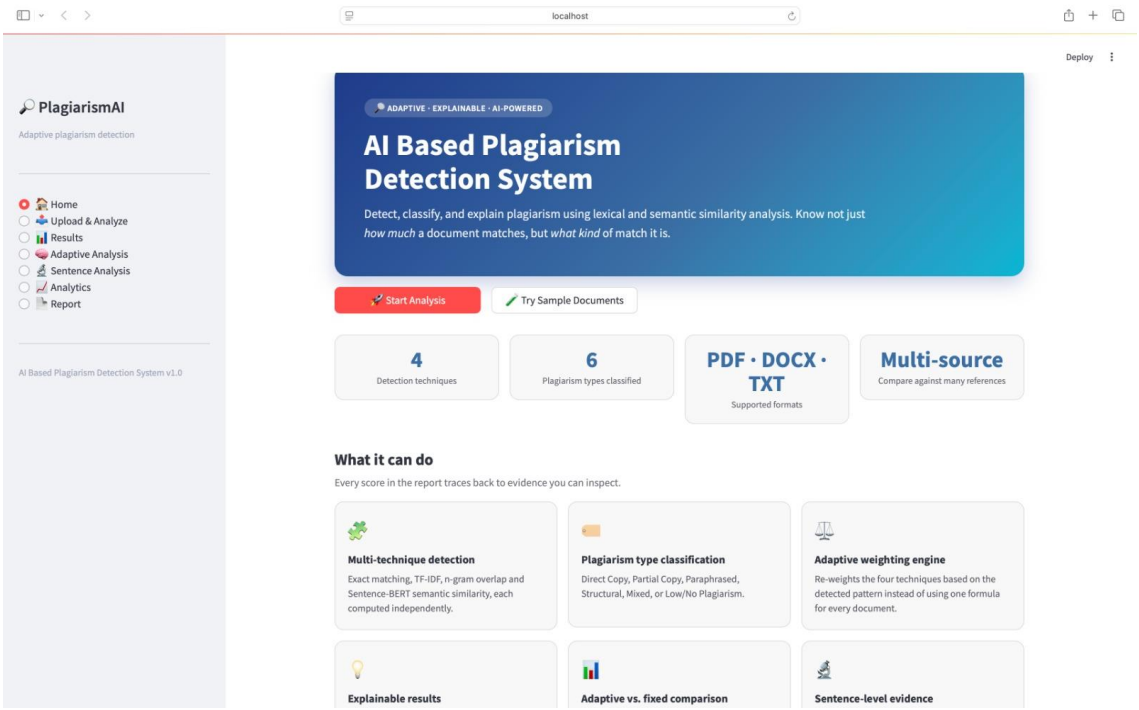


Figure 8.1: Home Page

8.4 Upload & Analyze Page

On the Upload & Analyze page (Figure 8.2) the user supplies the suspicious document in step 1 and the reference documents in step 2. Each step offers a choice between uploading files and using a bundled sample. The uploader states the limit of 200 MB per file and the accepted formats PDF, TXT and DOCX. In the recorded run doc1.txt (1.1 KB) was selected in both steps, and the “Run Adaptive Plagiarism Analysis” button starts the analysis.

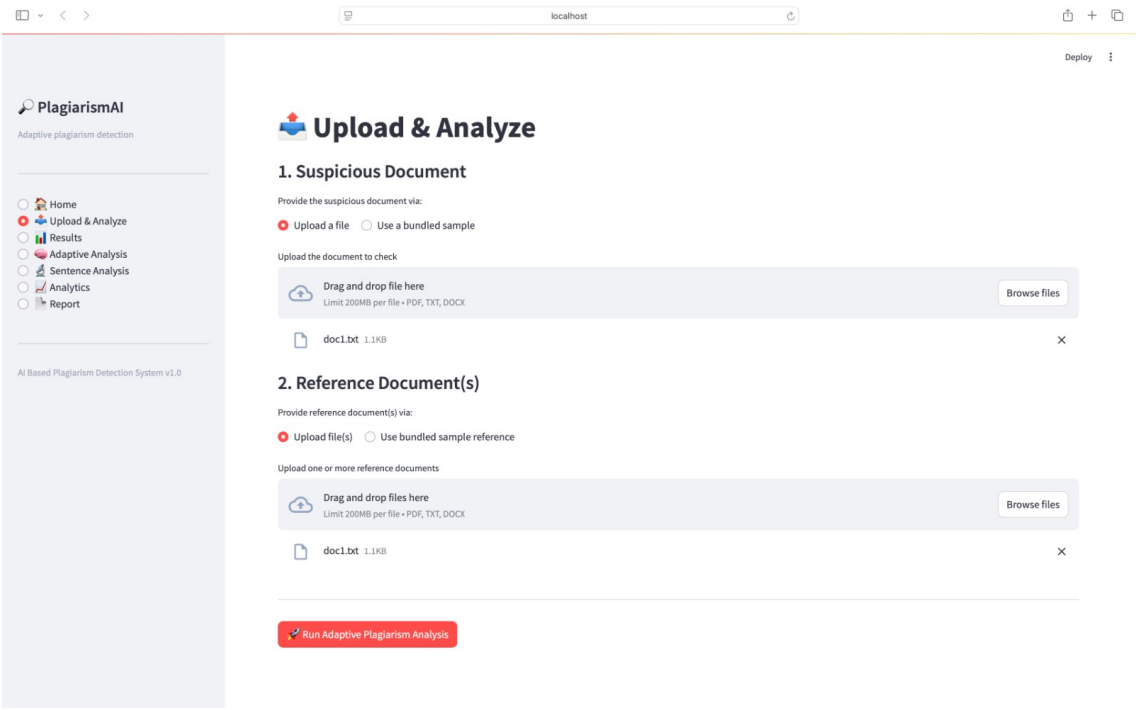


Figure 8.2: Upload & Analyze Page

8.5 Results Page

The Results page (Figure 8.3) shows five headline figures: the adaptive plagiarism score of 100.0%, the detected type Direct Copy, the classification confidence of 99.0%, the most similar source doc1.txt and 7 / 7 suspicious sentences. A red alert states that plagiarism is suspected against doc1.txt because the score is at least 40%. The “Detection Method Comparison” bar chart shows the four detector scores, all at 100.0%, and the same four values are repeated as figures below the chart.

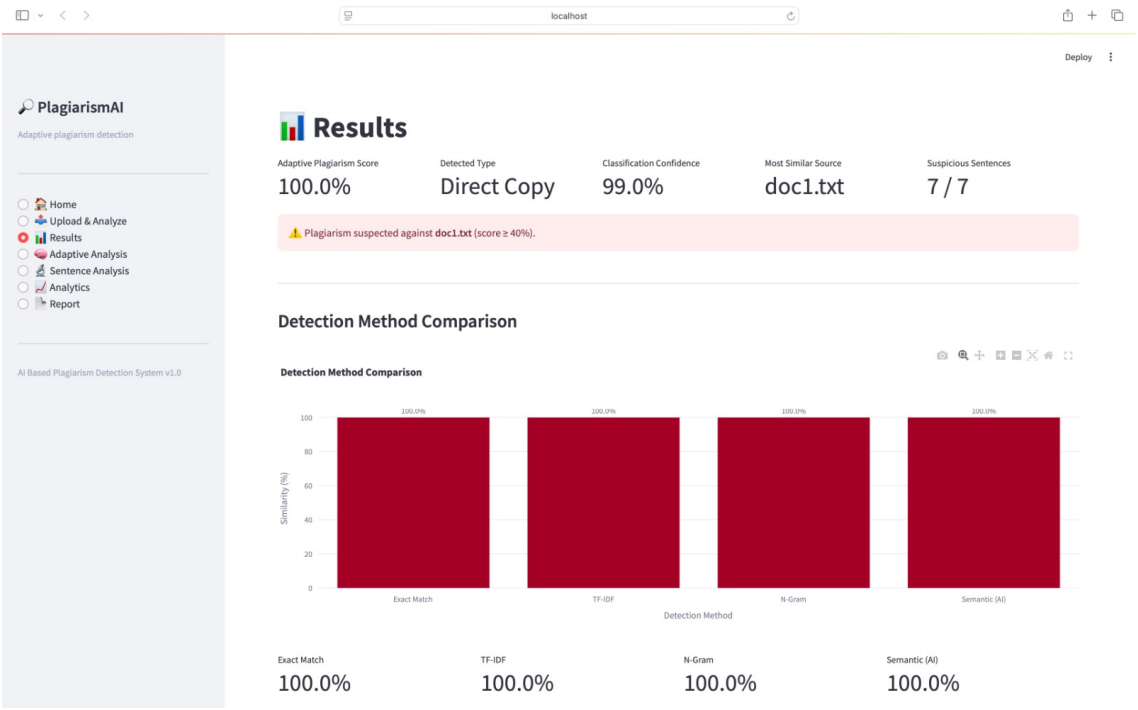


Figure 8.3: Results Page

8.6 Adaptive Analysis Page

The Adaptive Analysis page (Figure 8.4) begins with the radar chart “Similarity Pattern”, on which the four scores fill all four axes, giving the diamond shape that is typical of a direct copy. The metrics beneath it show the detected type Direct Copy, the classification confidence of 99.0%, the traditional fixed score of 100.0% and the adaptive score of 100.0%, with a change of +0.0.

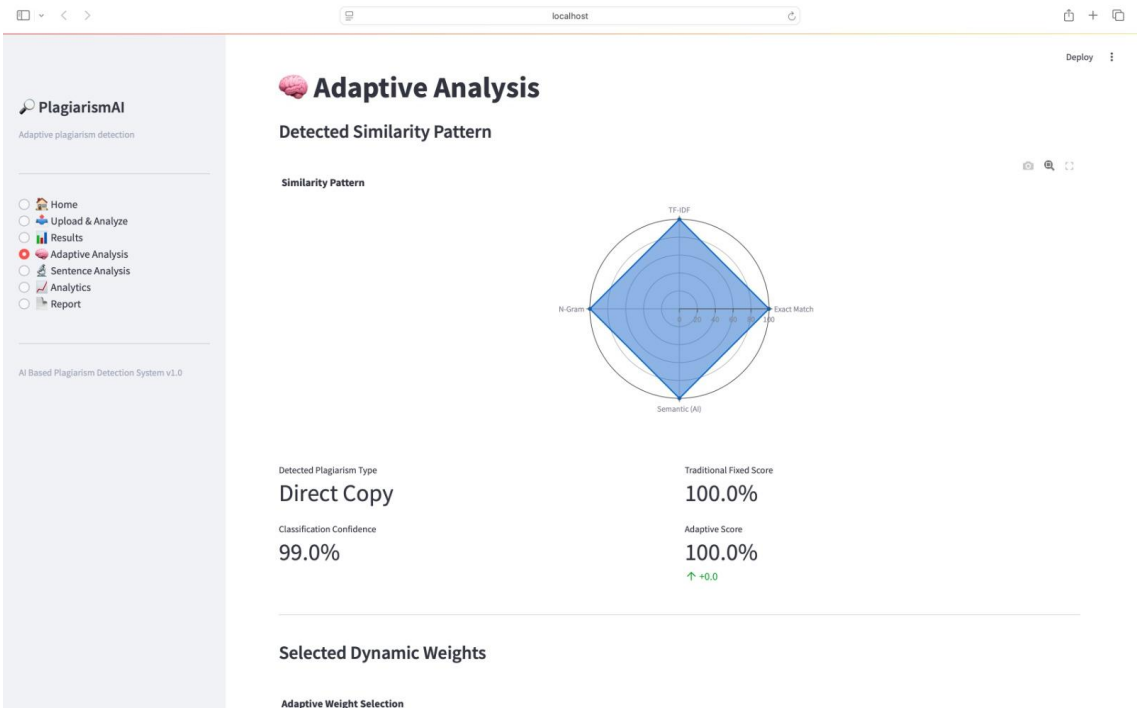


Figure 8.4: Adaptive Analysis Page: Similarity Pattern and Scores

The section “Selected Dynamic Weights” (Figure 8.5) shows the weights that the engine selected for the Direct Copy type: Exact Match 40%, TF-IDF 30%, N-gram 20% and Semantic 10%. The note under the chart states why: exact and lexical matching dominate because the content was copied near-verbatim.

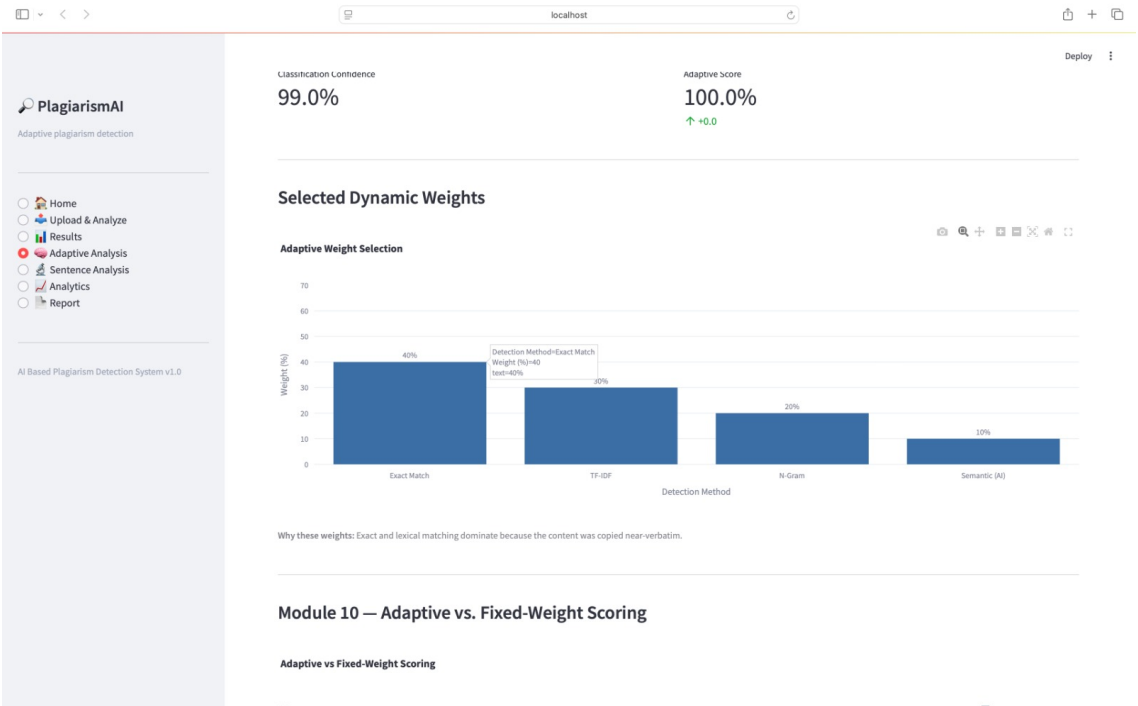


Figure 8.5: Selected Dynamic Weights

Module 10, “Adaptive vs. Fixed-Weight Scoring” (Figure 8.6), places the traditional fixed score and the adaptive score side by side; here both are 100%, so the bars are equal. A caption states the fixed baseline weights, Exact 20%, TF-IDF 25%, N-gram 20% and Semantic 35%. The “Explanation” section gives three numbered statements: the strongest signal was exact text similarity at 100.0%; 7 of 7 compared sentences were flagged (similarity ≥ 65%) with the strongest match at 100.0% against doc1.txt, classified as Exact / Near-Exact Copy; and the pattern matched Direct Copy with 99.0% confidence, so the detectors were weighted Exact 40%, TF-IDF 30%, N-gram 20% and Semantic 10%. A blue box concludes that the document contains substantial verbatim or near-verbatim text matching the reference documents, consistent with direct copying (score 100.0%).

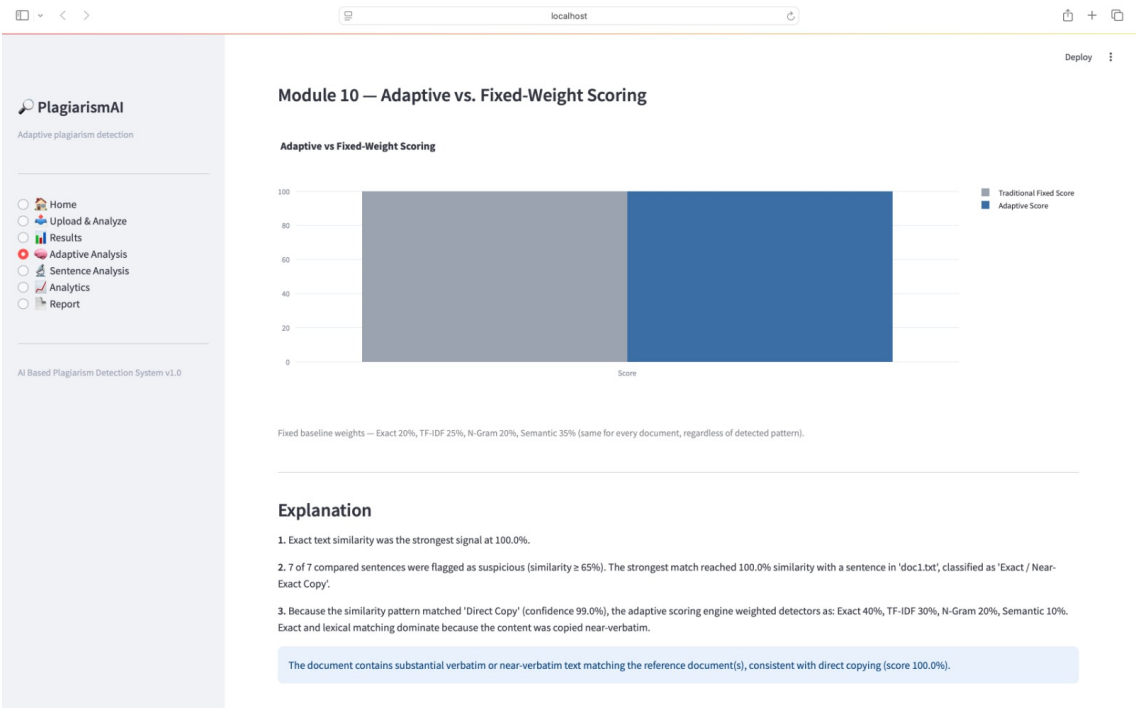


Figure 8.6: Adaptive vs. Fixed-Weight Scoring and Explanation

8.7 Sentence Analysis Page

The Sentence Analysis page (Figure 8.7) lists the seven compared sentences, each in an expandable row with a red marker, the similarity of 100.0% and the beginning of the sentence. Two drop-down lists filter the rows by similarity and by plagiarism type, and the caption “Showing 7 of 7 compared sentences” reports how many rows are visible.

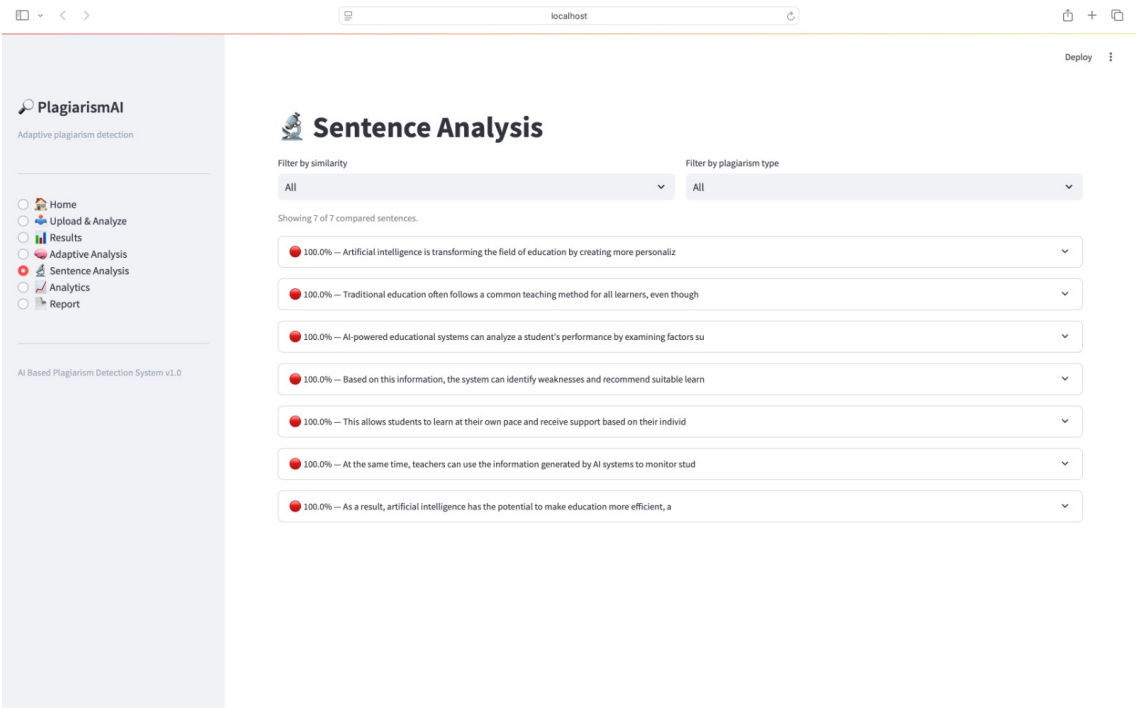


Figure 8.7: Sentence Analysis Page

8.8 Analytics Page

The Analytics Dashboard (Figure 8.8) repeats the five headline figures and arranges the charts in two columns: the detection method comparison (all four scores at 100.0%) beside the adaptive weight selection (40%, 30%, 20% and 10%).

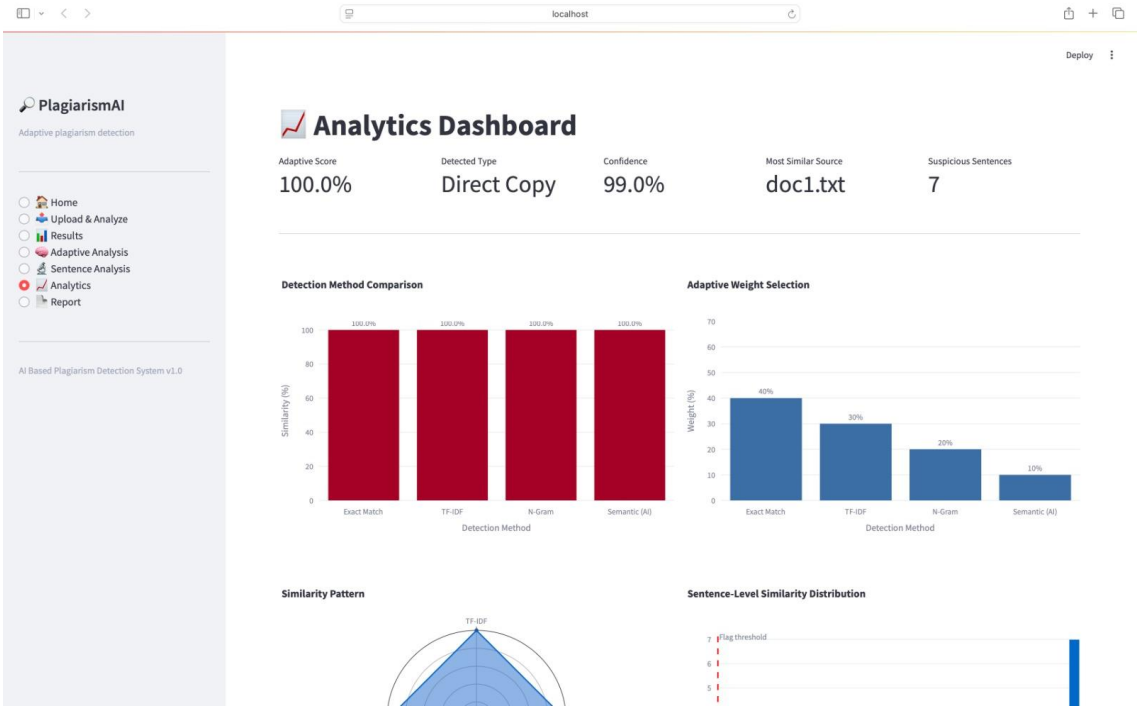


Figure 8.8: Analytics Dashboard: Metrics and Method Charts

The lower part of the dashboard (Figure 8.9) shows the similarity-pattern radar beside the sentence-level similarity distribution, in which all seven sentences fall in the highest bin, far to the right of the dashed flag threshold at 65%. The page ends with the "Plagiarism Type Summary": Direct Copy, confidence 99.0%.

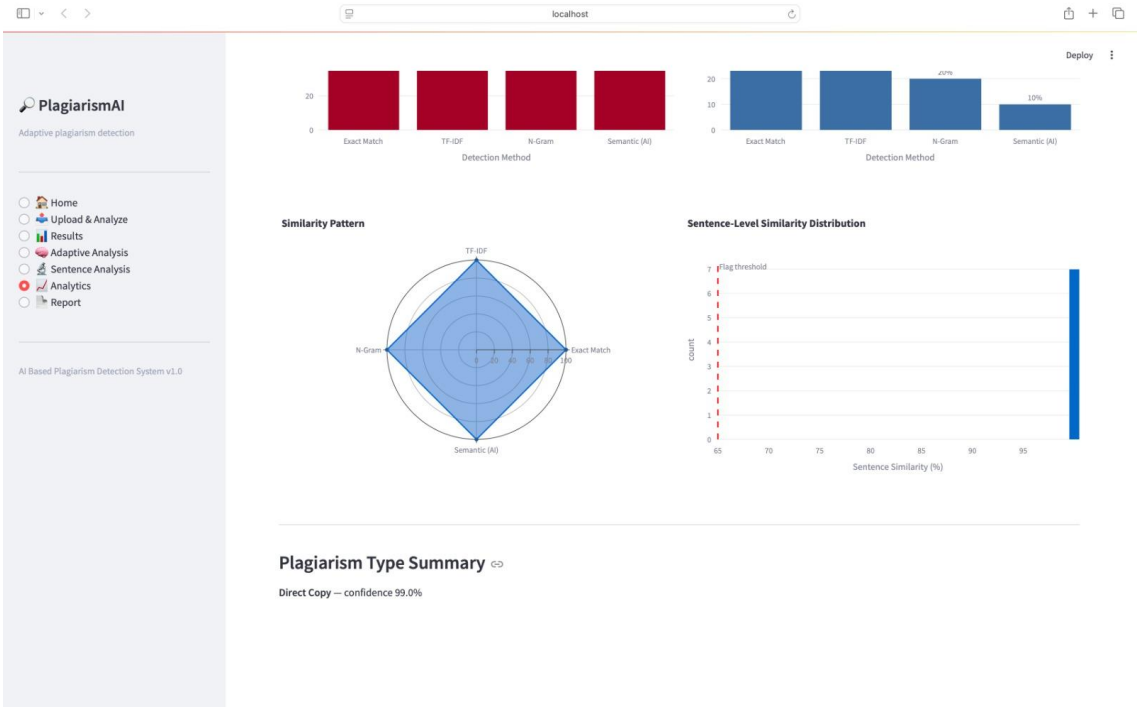


Figure 8.9: Analytics Dashboard: Pattern, Distribution and Type Summary

8.9 Report Page and PDF Report

The Report page (Figure 8.10) states that a downloadable PDF report can be generated for doc1.txt versus doc1.txt, including detection results, adaptive analysis, explainable findings and suspicious sentence evidence. The report is created when the "Generate PDF Report" button is pressed.

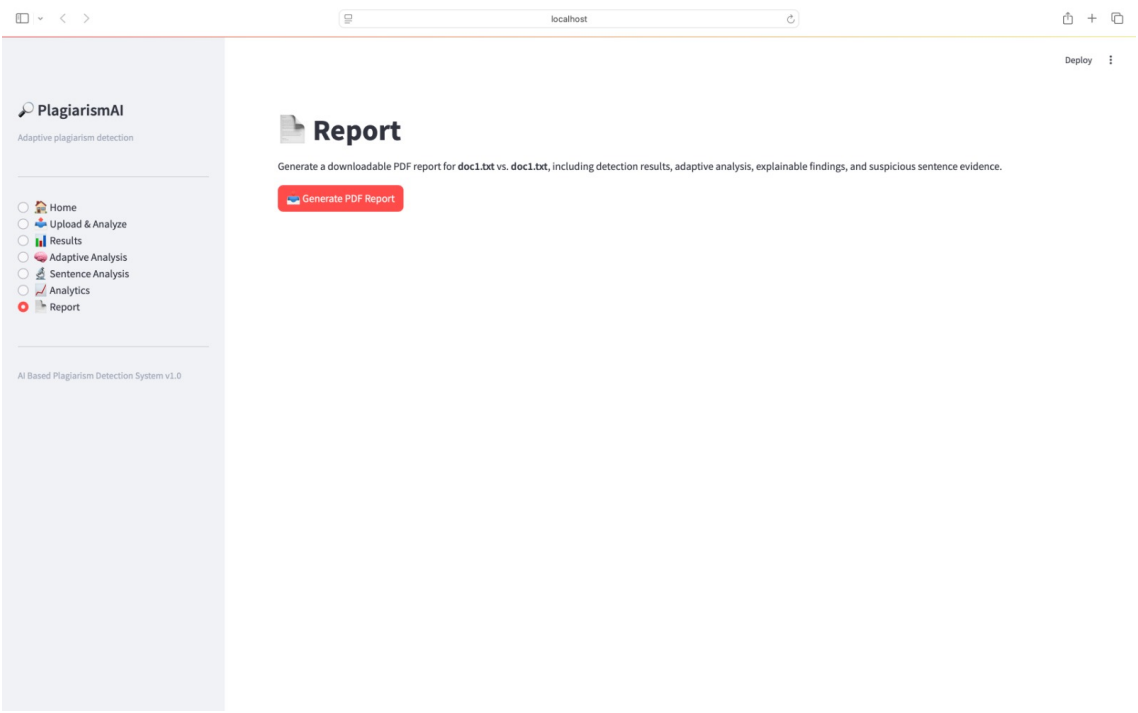
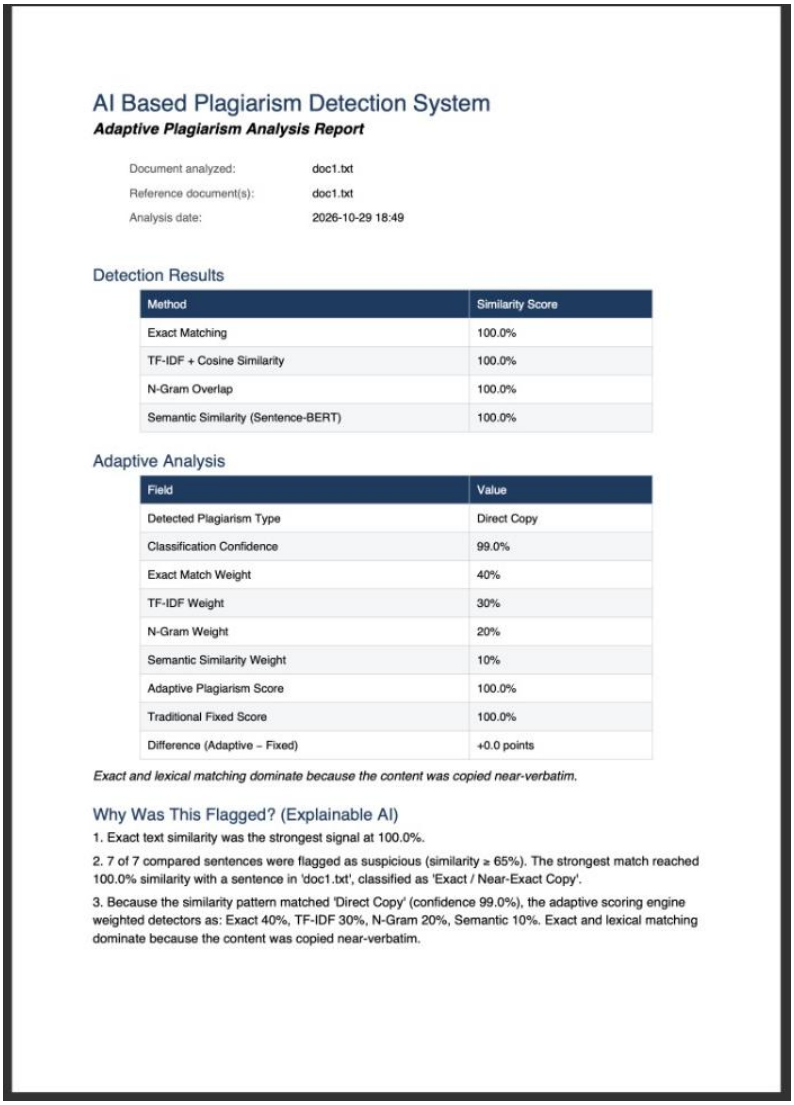


Figure 8.10: Report Page

The first page of the PDF (Figure 8.11) carries the title “AI Based Plagiarism Detection System: Adaptive Plagiarism Analysis Report”, the analysed document, the reference documents and the analysis date and time (2026-10-29 18:49). It contains the Detection Results table with the four methods at 100.0% each, the Adaptive Analysis table with the type, confidence, the four weights, the adaptive and fixed scores and the difference of +0.0 points, and the section “Why Was This Flagged? (Explainable AI)” with the three explanation statements.



AI Based Plagiarism Detection System

Adaptive Plagiarism Analysis Report

Document analyzed: doc1.txt
Reference document(s): doc1.txt
Analysis date: 2026-10-29 18:49

Detection Results

Method	Similarity Score
Exact Matching	100.0%
TF-IDF + Cosine Similarity	100.0%
N-Gram Overlap	100.0%
Semantic Similarity (Sentence-BERT)	100.0%

Adaptive Analysis

Field	Value
Detected Plagiarism Type	Direct Copy
Classification Confidence	99.0%
Exact Match Weight	40%
TF-IDF Weight	30%
N-Gram Weight	20%
Semantic Similarity Weight	10%
Adaptive Plagiarism Score	100.0%
Traditional Fixed Score	100.0%
Difference (Adaptive − Fixed)	+0.0 points

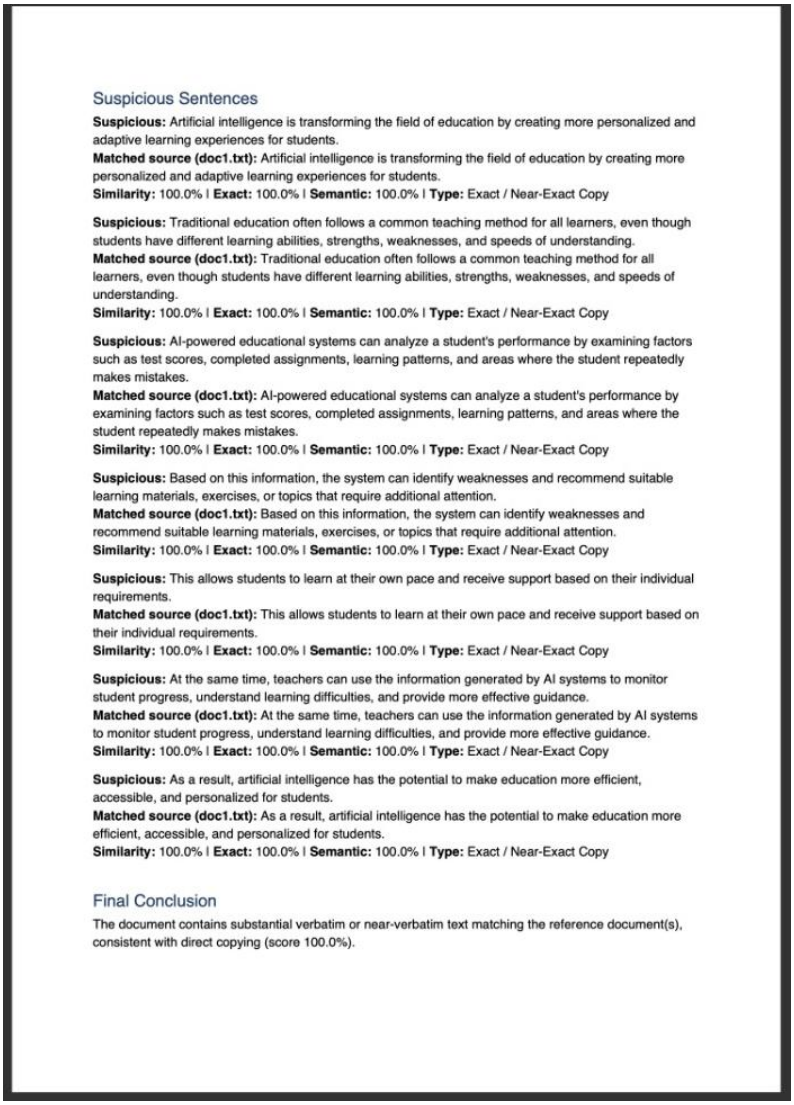
Exact and lexical matching dominate because the content was copied near-verbatim.

Why Was This Flagged? (Explainable AI)

1. Exact text similarity was the strongest signal at 100.0%.
2. 7 of 7 compared sentences were flagged as suspicious (similarity ≥ 65%). The strongest match reached 100.0% similarity with a sentence in 'doc1.txt', classified as 'Exact / Near-Exact Copy'.
3. Because the similarity pattern matched 'Direct Copy' (confidence 99.0%), the adaptive scoring engine weighted detectors as: Exact 40%, TF-IDF 30%, N-Gram 20%, Semantic 10%. Exact and lexical matching dominate because the content was copied near-verbatim.

Figure 8.11: PDF Report, Page 1

The second page (Figure 8.12) lists the seven suspicious sentences. For each, the report prints the suspicious sentence, the matched source sentence from doc1.txt, and the similarity, exact and semantic values, all 100.0%, together with the type label Exact / Near-Exact Copy. The page ends with the Final Conclusion, which states that the document contains substantial verbatim or near-verbatim text matching the reference documents, consistent with direct copying (score 100.0%).



Suspicious Sentences

Suspicious: Artificial intelligence is transforming the field of education by creating more personalized and adaptive learning experiences for students.
Matched source (doc1.txt): Artificial intelligence is transforming the field of education by creating more personalized and adaptive learning experiences for students.
Similarity: 100.0% | **Exact:** 100.0% | **Semantic:** 100.0% | **Type:** Exact / Near-Exact Copy

Suspicious: Traditional education often follows a common teaching method for all learners, even though students have different learning abilities, strengths, weaknesses, and speeds of understanding.
Matched source (doc1.txt): Traditional education often follows a common teaching method for all learners, even though students have different learning abilities, strengths, weaknesses, and speeds of understanding.
Similarity: 100.0% | **Exact:** 100.0% | **Semantic:** 100.0% | **Type:** Exact / Near-Exact Copy

Suspicious: AI-powered educational systems can analyze a student's performance by examining factors such as test scores, completed assignments, learning patterns, and areas where the student repeatedly makes mistakes.
Matched source (doc1.txt): AI-powered educational systems can analyze a student's performance by examining factors such as test scores, completed assignments, learning patterns, and areas where the student repeatedly makes mistakes.
Similarity: 100.0% | **Exact:** 100.0% | **Semantic:** 100.0% | **Type:** Exact / Near-Exact Copy

Suspicious: Based on this information, the system can identify weaknesses and recommend suitable learning materials, exercises, or topics that require additional attention.
Matched source (doc1.txt): Based on this information, the system can identify weaknesses and recommend suitable learning materials, exercises, or topics that require additional attention.
Similarity: 100.0% | **Exact:** 100.0% | **Semantic:** 100.0% | **Type:** Exact / Near-Exact Copy

Suspicious: This allows students to learn at their own pace and receive support based on their individual requirements.
Matched source (doc1.txt): This allows students to learn at their own pace and receive support based on their individual requirements.
Similarity: 100.0% | **Exact:** 100.0% | **Semantic:** 100.0% | **Type:** Exact / Near-Exact Copy

Suspicious: At the same time, teachers can use the information generated by AI systems to monitor student progress, understand learning difficulties, and provide more effective guidance.
Matched source (doc1.txt): At the same time, teachers can use the information generated by AI systems to monitor student progress, understand learning difficulties, and provide more effective guidance.
Similarity: 100.0% | **Exact:** 100.0% | **Semantic:** 100.0% | **Type:** Exact / Near-Exact Copy

Suspicious: As a result, artificial intelligence has the potential to make education more efficient, accessible, and personalized for students.
Matched source (doc1.txt): As a result, artificial intelligence has the potential to make education more efficient, accessible, and personalized for students.
Similarity: 100.0% | **Exact:** 100.0% | **Semantic:** 100.0% | **Type:** Exact / Near-Exact Copy

Final Conclusion

The document contains substantial verbatim or near-verbatim text matching the reference document(s), consistent with direct copying (score 100.0%).

Figure 8.12: PDF Report, Page 2

8.10 Summary of the Sample Run

Table 8.1: Summary of the Sample Run

Item	Value
Suspicious document / reference	doc1.txt / doc1.txt
Detector scores (Exact, TF-IDF, N-gram, Semantic)	100.0%, 100.0%, 100.0%, 100.0%
Detected type and confidence	Direct Copy, 99.0%
Weights applied	Exact 40%, TF-IDF 30%, N-gram 20%, Semantic 10%
Adaptive score / fixed score	100.0% / 100.0% (difference +0.0)
Suspicious sentences	7 of 7 (threshold 65%)
Alert	Plagiarism suspected against doc1.txt (score ≥ 40%)
PDF report	Two pages, generated on 2026-10-29 18:49

The output shows that every page reads the same result and that the explanation, the charts and the PDF are consistent with each other. Because the sample compares a document with itself,

it demonstrates the Direct Copy case and the complete workflow; the behaviour on reworded text is to be shown with the further tests of Chapter 7.

APPENDIX A

USER MANUAL

A.1 Purpose

This appendix explains how to use PlagiarismAI without reading the rest of the report. It is written for students who want to check their own work and for faculty who want to screen submissions.

A.2 Starting the Application

1. Open a terminal in the project folder and start the application with the Streamlit command for the main program file. The code can be cloned from GitHub (see Appendix B).
2. Open the address shown in the terminal (localhost) in a web browser.
3. The Home page appears. The navigation menu on the left lists the seven pages.

A.3 Checking a Document

1. Press “Start Analysis” on the Home page, or choose Upload & Analyze in the menu. To try the system first, press “Try Sample Documents”.
2. In step 1, choose “Upload a file” and select the suspicious document (PDF, TXT or DOCX), or choose “Use a bundled sample”.
3. In step 2, choose “Upload file(s)” and select one or more reference documents, or choose “Use bundled sample reference”.
4. Press “Run Adaptive Plagiarism Analysis”. The first run is slower because the embedding model is loaded.
5. Open the Results page to see the score, the type and the alert, then the other pages for the details.

A.4 Reading the Result

Table A.1: Elements of the Result Pages

Element	What it tells the reader
Adaptive Plagiarism Score	The weighted combination of the four detector scores, using the weights of the detected type
Detected Type	Direct Copy, Partial Copy, Paraphrased, Structural, Mixed or Low / No Plagiarism, decided from the score pattern
Classification Confidence	How strongly the score pattern fits the detected type
Most Similar Source	The reference document with the highest adaptive score
Suspicious Sentences	Number of sentences with similarity of at least 65%, out of all compared sentences
Alert	Shown when the score reaches 40%

Element	What it tells the reader
Detection Method Comparison	The score of each technique; a large gap between lexical and semantic scores indicates rewording
Similarity Pattern (radar)	The shape of the four scores; a full diamond indicates a copy
Selected Dynamic Weights	How much each technique contributed, which depends on the detected type
Adaptive vs. Fixed score	The adaptive score beside the traditional fixed-weight score and their difference
Explanation	Three statements that say why the document was flagged
Sentence Analysis	Sentence pairs that are very similar; the evidence to check by eye

A.5 Sentence Analysis, Analytics and Reports

On Sentence Analysis, open a row to read the suspicious sentence and its matched source, and use the two filters to show only sentences above a similarity level or of one plagiarism type. The Analytics page shows all charts on one screen. On the Report page press “Generate PDF Report” to create a two-page report of the analysis, and download it. The result is not stored after the session ends, so download the report if a record is needed.

A.6 Guidance for Reviewers

- A high score is a reason to read the matching sentences, not a conclusion. Quotations, standard definitions and common technical phrases also produce similarity.
- If the reference is a source that the submission cites properly, similarity is expected and is not misconduct.
- A low score does not prove originality. The system compares only with the references that were supplied.
- Semantic scores can be moderate for texts that discuss the same topic independently. Look at the matching sentences before drawing a conclusion.

A.7 Troubleshooting

Table A.2: Common Problems and Solutions

Problem	Likely cause	Solution
The result pages ask to run an analysis	No analysis has been run in this session	Go to Upload & Analyze and run the analysis
A file is not accepted	Unsupported type or larger than 200 MB	Use a PDF, TXT or DOCX file within the limit
Error that no text could be read	A scanned PDF without a text layer, or an empty file	Use a text-based PDF or convert the file to DOCX or TXT
First analysis is slow	The embedding model is loading	Wait; later analyses are faster because the model stays in memory
Page does not open	Wrong address or port in use	Use the address shown in the terminal, or start on another port

APPENDIX B

SOURCE CODE REPOSITORY

B.1 Repository Details

The complete source code of PlagiarismAI, together with its configuration file, the bundled sample documents and the test scripts, is published as a public repository on GitHub [23]. The repository can be opened in a web browser, downloaded as an archive or cloned with Git using the details in Table B.1.

Table B.1: Details of the Source Code Repository

Item	Details
Repository name	my-streamlit-project
Owner	Ace99-coder
Repository link	https://github.com/Ace99-coder/my-streamlit-project
Clone address	https://github.com/Ace99-coder/my-streamlit-project.git
Default branch	main
Visibility	Public
Main program file	app.py (Streamlit entry point of the seven application pages)

B.2 Repository Structure

The detectors, the plagiarism-type classifier, the adaptive weighting logic, the explanation engine and the report generator described in Chapters 3 and 4 are kept in the modules folder, and the seven application pages are implemented in app.py. Table B.2 lists the folders and files of the repository.

Table B.2: Folders and Files of the Repository

Folder / File	Contents
app.py	Streamlit entry point that builds the seven pages of the application
requirements.txt	Python packages needed to run the application
README.md	Project description, architecture, installation steps and module overview
config/	config.yaml, which holds the classification thresholds and the adaptive weight profiles
data/	Bundled sample documents and a reference document, and the folder used for uploads and generated reports
database/	database.py and models.py: SQLite connection and table definitions

Folder / File	Contents
modules/	Document processing, preprocessing, the four detectors, classifier, adaptive weighting, hybrid scorer, sentence analysis, explanation engine and PDF report generator
utils/	Helper functions and validators for the configuration and the weight profiles
tests/	Test scripts for the similarity detectors, the classifier and the adaptive weighting

B.3 Installation and Execution

The application runs on any computer with Python 3.10 or later and Git. The commands below clone the repository, install the dependencies and start the application.

```
# 1. Clone the repository
git clone https://github.com/Ace99-coder/my-streamlit-project.git
cd my-streamlit-project

# 2. Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate        (Windows: venv\Scripts\activate)

# 3. Install the dependencies
pip install -r requirements.txt

# 4. Download the NLTK data used for preprocessing
python3 -m nltk.downloader punkt stopwords wordnet punkt_tab

# 5. Start the application
streamlit run app.py
```

Streamlit prints the address of the application in the terminal, usually http://localhost:8501. Open it in a web browser and follow the steps of Appendix A. The first analysis downloads the Sentence-BERT model (all-MiniLM-L6-v2, about 90 MB), so an internet connection is needed once; after that the model is stored locally.

B.4 Running the Tests

The tests folder contains test scripts for the detectors, the classifier and the adaptive weighting. They can be run from the project folder with the following command, or by running each script directly with Python.

```
pytest tests/ -v
```

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project designed and implemented PlagiarismAI, an Adaptive AI-Based Plagiarism Detection System that goes beyond a single similarity percentage. The system combines four complementary techniques (exact matching, TF-IDF cosine similarity, n-gram overlap and Sentence-BERT semantic similarity), classifies the likely type of plagiarism into one of six types from the pattern of their scores, adapts the weights of the detectors to that type and compares the resulting adaptive score with a traditional fixed-weight score. It accepts PDF, DOCX and TXT files and presents the outcome through seven pages ending in a PDF report. The literature review showed that lexical methods are precise but fail on reworded text, while semantic methods detect reworded text but can overstate topical similarity; the adaptive design addresses this by choosing which evidence to trust and by showing the reviewer the scores, the weights, a plain-language explanation and the sentences that caused the result.

In the recorded run, in which a seven-sentence document was compared with itself, all four detectors returned 100%, the classifier reported Direct Copy with a confidence of 99.0%, the weights 40%, 30%, 20% and 10% were applied, all seven sentences were flagged, and a two-page PDF report was generated. The run confirms that the complete pipeline works and that all pages and the report agree.

9.2 Limitations

1. **Small evaluation.** Only one run, a document compared with itself, has been recorded. It cannot show the advantage of adaptive weighting, and no accuracy, precision or recall has been measured on a labelled corpus.
2. **Heuristic parameters.** The classification thresholds, the weight profiles and the alert and sentence thresholds (40% and 65%) are design parameters that have not been calibrated from data.
3. **Reference text must be supplied.** The system compares a submission with the references given by the user or bundled with it; it does not search the web or a large index.
4. **Input and language limits.** Scanned PDFs without a text layer cannot be read, only English text is supported, and source code, images and AI-generated text are not analysed.
5. **Semantic scores need interpretation.** Embedding similarity can be moderate for texts that merely discuss the same topic.
6. **Prototype scope.** Results are kept only in the browser session (the PDF report is the only record), and there is no login or concurrent-use support.

9.3 Future Scope

- **Calibration on labelled data.** Evaluate the system on a labelled corpus such as that of Clough and Stevenson [5], report accuracy, precision, recall, F1-score and a confusion matrix, tune the thresholds and learn the weights with a classifier such as logistic regression.
- **Larger reference collections and web sources.** Compare a submission with whole collections using a fingerprint index [11] and precomputed embeddings, and integrate search or open-access repositories.
- **History, highlighting and OCR.** Store earlier analyses, highlight matching passages inside the document and read scanned PDFs with optical character recognition.
- **Multilingual and cross-language detection** with multilingual embedding models and language-specific preprocessing.
- **Source code and portfolio analysis** so that resume and portfolio claims can be checked against the actual code and repository structure.
- **AI-generated text awareness** through complementary signals such as stylometric analysis and viva-based verification, with attention to their reliability.
- **Authentication, API and deployment.** A REST API, role-based access, batch screening of a whole class and integration with a learning management system.
- **Reviewer feedback.** Reviewers could mark matches as correct or incorrect, producing labelled data to improve the thresholds and weights while the final decision stays with a human.

9.4 Summary

The project achieved its objective of building an adaptive, explainable plagiarism detector with a working engine, a seven-page interface, analytics and PDF reporting. The main steps that remain are evaluation on reworded and labelled text and calibration of the adaptive parameters, the first items of the future scope.

REFERENCES

[1] H. Maurer, F. Kappe, and B. Zaka, “Plagiarism – A survey,” J. Universal Computer Science, vol. 12, no. 8, pp. 1050–1084, 2006.

[2] S. M. Alzahrani, N. Salim, and A. Abraham, “Understanding plagiarism linguistic patterns, textual features, and detection methods,” IEEE Trans. Systems, Man, and Cybernetics, Part C (Applications and Reviews), vol. 42, no. 2, pp. 133–149, Mar. 2012.

[3] T. Foltýnek, N. Meuschke, and B. Gipp, “Academic plagiarism detection: A systematic literature review,” ACM Computing Surveys, vol. 52, no. 6, Art. no. 112, 2019.

[4] M. Potthast, B. Stein, A. Eiselt, A. Barrón-Cedeño, and P. Rosso, “Overview of the 1st International Competition on Plagiarism Detection,” in Proc. SEPLN 2009 Workshop on Uncovering Plagiarism, Authorship and Social Software Misuse (PAN 09), 2009.

[5] P. Clough and M. Stevenson, “Developing a corpus of plagiarised short answers,” Language Resources and Evaluation, vol. 45, no. 1, pp. 5–24, 2011.

[6] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[7] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. Conf. Empirical Methods in Natural Language Processing and Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3982–3992.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[9] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” arXiv:1301.3781, 2013.

[10] J. Pennington, R. Socher, and C. D. Manning, “GloVe: Global vectors for word representation,” in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2014, pp. 1532–1543.

[11] S. Schleimer, D. S. Wilkerson, and A. Aiken, “Winnowing: Local algorithms for document fingerprinting,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2003, pp. 76–85.

[12] A. Z. Broder, “On the resemblance and containment of documents,” in Proc. Compression and Complexity of Sequences (SEQUENCES), IEEE, 1997, pp. 21–29.

[13] O. Lukashenko, V. Graudina, and J. Grundspenkis, “Computer-based plagiarism detection methods and tools: An overview,” in Proc. Int. Conf. Computer Systems and Technologies (CompSysTech), 2007.

[14] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[15] A. Vaswani et al., “Attention is all you need,” in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 5998–6008.

[16] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. Sebastopol, CA, USA: O’Reilly Media, 2009.

[17] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in Advances in Neural Information Processing Systems 33 (NeurIPS), 2020, pp. 5776–5788.

[18] M. Potthast, B. Stein, A. Barrón-Cedeño, and P. Rosso, “An evaluation framework for plagiarism detection,” in Proc. 23rd Int. Conf. Computational Linguistics (COLING): Posters, 2010, pp. 997–1005.

[19] spaCy, “Industrial-strength natural language processing in Python.” [Online]. Available: https://spacy.io

[20] Streamlit Inc., “Streamlit documentation.” [Online]. Available: https://docs.streamlit.io

[21] ReportLab, “ReportLab documentation.” [Online]. Available: https://docs.reportlab.com

[22] Plotly Technologies Inc., “Plotly Python graphing library documentation.” [Online]. Available: https://plotly.com/python/

[23] S. Das, A. Kumar, and D. Kumar V, “PlagiarismAI: Adaptive AI-based plagiarism detection system,” GitHub repository, 2026. [Online]. Available: https://github.com/Ace99-coder/my-streamlit-project

IRE JOURNALS

Iconic Research and Engineering Journals

International Peer-Reviewed Open Access Journal

October 2026

ISSN : 2456-8880

PAPER ID
IRE-1723909

DATE
09 Oct 2026

Acceptance Letter

Dear Ayush Kumar,

We are pleased to inform you that your research paper titled **Adaptive AI-Based Plagiarism Detection System** (Paper ID: **1723909**) has been successfully reviewed and formally accepted for publication in **IRE Journals, Volume 10 / Issue 4, October - 2026** .

Your manuscript underwent a rigorous evaluation by our editorial board and external reviewers, assessed based on its originality, technical clarity, contextual relevance, and meaningful contribution to its respective field of study. The reviewers found your work to be of high scholarly merit and a valuable addition to our multidisciplinary academic repository.

We sincerely appreciate your scholarly contribution and look forward to showcasing your research in our upcoming issue. Once the publication process is complete, you will receive the official e-certificate of publication and the permanent link to your published paper.

Thank you for choosing IRE Journals as the platform to share your research. We wish you continued success in your academic and professional journey.

With best regards,

Sanket Patel

Mr. Sanket Patel

Editor-in-Chief

IRE Journals

Certificates of Publication

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

SAURAV DAS

has published the research paper entitled

ADAPTIVE AI-BASED PLAGIARISM DETECTION SYSTEM

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723909

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

AYUSH KUMAR

has published the research paper entitled

ADAPTIVE AI-BASED PLAGIARISM DETECTION SYSTEM

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723909

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

DEEPAK KUMAR V

has published the research paper entitled

ADAPTIVE AI-BASED PLAGIARISM DETECTION SYSTEM

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723909

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

DEEPTI N N

has published the research paper entitled

ADAPTIVE AI-BASED PLAGIARISM DETECTION SYSTEM

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723909

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

Adaptive AI-Based Plagiarism Detection System

Ayush Kumar[1], Deepak Kumar V[2], Saurav das[3], Deepti N N(Assistant professor)[4]

[1]*Department of Computer Science and Engineering, Rajiv Gandhi Institute of Technology, Visvesvaraya Technological University, Bengaluru, India, ayush0835@gamil.com*

[2]*Department of Computer Science and Engineering, Rajiv Gandhi Institute of Technology, Visvesvaraya Technological University, Bengaluru, India, astralknight216@gmail.com*

[3]*Department of Computer Science and Engineering, Rajiv Gandhi Institute of Technology, Visvesvaraya Technological University, Bengaluru, India, 20022000sauravv@gmail.com*

[4]*Department of Computer Science and Engineering, Rajiv Gandhi Institute of Technology, Visvesvaraya Technological University, Bengaluru, India, nndeepti@gmail.com*

Abstract

Traditional plagiarism detection systems normally compute a separate similarity score applying a corrected combination of detection techniques, regardless of the type of plagiarism presence. This approach performs poorly when the nature of text-based overlap varies; for instance, verbatim copying produces high scores across all word-related measures, whereas rephrasing produces low word-related similarity but high meaning-related similarity. This paper presents an adaptive plagiarism detection system that first computes four unaffiliated similarity measures; exact sequence correspondence, TF-IDF cosine similarity, n-gram overlap, and Sentence-BERT meaning-related similarity; then classifies the anticipated plagiarism pattern utilizing a customizable, rule-based classifier, and ultimately re-weights the four measures according to the detected pattern before computing a last and ultimate adaptive score. The system moreover offers sentence-level evidence and a traceable, non-templated explanation.

Keywords: *plagiarism detection, natural language processing, adaptive weighting, semantic similarity, Sentence-BERT, TF-IDF, explainable AI*

I. Introduction

Because of the increasing amount of digital content being produced in educational and inquiry environments, there is now a need for automated plagiarism detection. Yet the majority of plagiarism detection tools mainly rely on exact or near-exact string matching and thus are unable to pick up on material that has been rephrased or semantically restructured in such a way that the same meaning is conveyed using different terms [1]. Even those systems that combine a number of similarity techniques generally do so by using a single correction formula that is applied in the same way to all pairs of documents, thereby assuming that all kinds of plagiarism produce the same numerical fingerprint. This assumption is not valid in reality: direct copying leads to high similarity scores across all three types of measures—exact, word-related, and meaning-related—while deliberate rewording results in low word-based similarity but still maintains high meaning-related similarity. The present paper proposes and investigates a system that

1. A rule-based, configuration-driven classifier that categorizes document-pair similarity into six patterns (Direct Copy, Partial Copy, Paraphrased Plagiarism, Structural Similarity, Mixed or Complex Similarity, and Low or No Significant Plagiarism) without requiring labelled training data.

2. An adaptive weighting engine that maps each detected pattern to an empirically motivated weight vector across four detection techniques, confirmed to invariably sum to unity.

3. A direct, controlled comparison between adaptive and fixed-weight scoring that isolates and shows the effect of adaptive weighting.

4. An explainability layer that generates natural-language justifications traceable to precise and particular computed values, and sentence-level evidence identifying which origin sentences contributed to a flagged score.

5. An experimental calibration of classification thresholds against measured baseline similarity between separate documents, alternatively than supposed values.

II. Related Work

Early versions of plagiarism detection systems like MOSS used string and token matching to identify plagiarised contents . They failed to detect and compare similar contnents if it was written in a different way. New models tried to compare semantically similar text that was written in different way. They represented text using dense terms like GloVe, and instead of comparing the symbols used in the text, they compared their related meanings. These models formed the basis of sentence and document level meaning comparison. Sentence-BERT uses transformers and is able to create meaningful sentence representations. It uses cosine-similarity to compare semantically similar documents. Models that rely on a single approach for detection have become way more easy to bypass. This approach inspired the use of multiple detectors in the current system. These models focused on adversarial attacks to copyright and plagiarism detection systems.

III. Proposed System Architecture

System Architecture

The system uses a multi-stage pipeline approach for adaptive plagiarism detection and for providing explanations in its reports. It accepts documents in PDF, DOCX, and TXT format. The document processing section extracts the textual content from the uploaded files and at the same time keeps the original sentence-level text ready for use when generating evidence. When carrying out preprocessing, the text is divided into tokens and cleaned. In order to carry out lexical comparison, the processed text is changed to lowercase, the stop words are deleted, and the tokens are lemmatized. The original sentence-level text is kept separately so that it can be used for sentence-level analysis and for exact verbatim comparisons.

The system then applies four methods for detecting plagiarism separately to both the suspicious document and the reference documents, normalising their outputs to a common scale from 0 to 100. The first of these is an exact-match detector which makes use of Python's difflib.SequenceMatcher to compare suspicious sentences with reference sentences and determines the most matching sentence for each comparison. The second technique is a TF-IDF detector that generates vector representations using scikit-learn and then calculates cosine similarity to assess lexical similarity. The third is an n-gram overlap detector that combines the unigram, bigram and trigram overlap coefficients in order to detect similarities in sequences of words. The fourth method is a semantic similarity detector that employs the Sentence-BERT model all-MiniLM-L6-v2 and computes cosine similarity in order to identify text that is semantically similar, including paraphrased material. If the transformer model cannot be loaded, for instance because of no internet connection or because the model resources are not available, the system falls back on a character n-gram TF-IDF as a substitute, thus enabling the pipeline to keep running rather than failing entirely.

The results from the four detectors are then fed into the plagiarism type classification module, where a rule-based classifier assigns the document to one of six plagiarism categories by means of threshold rules that are defined in an external configuration file. These rules are checked in order of priority until a matching category is found. Once the type of plagiarism has been detected, the adaptive weight selection mechanism chooses the appropriate weight profile. The weights selected take into account the expected importance of each similarity measure for the particular plagiarism pattern in question; for

instance, exact matching is given the highest weight in the case of direct copying, while semantic similarity is assigned a higher weight when dealing with paraphrased plagiarism. The adaptive plagiarism score is then computed as a weighted sum of the four detector scores.

The weight profiles associated with each type of plagiarism detected are given in Table I, and the system also has a fixed baseline weight profile for comparison with the adaptive weighting method. When the plagiarism level is low or none is detected, equal weights are given to all four detectors in order not to favour any particular similarity measure in the absence of a predominant plagiarism pattern.

The module that carries out the sentence-level analysis then finds, for each suspicious sentence, the reference sentence that matches it most closely and keeps the corresponding similarity sub-scores as evidence. This information is passed on to the explainable AI reporting module, which creates natural-language justifications that are traceable and are based entirely on the evidence and the numerical values calculated in the previous stages. The final results—comprising the plagiarism category identified, the adaptive plagiarism score, the sentence-level evidence, and the explanations—are displayed via a Streamlit dashboard and can also be exported in the form of a downloadable PDF report.

Figure 1. A diagram showing the system architecture of the proposed adaptive plagiarism detection and explainable reporting framework..

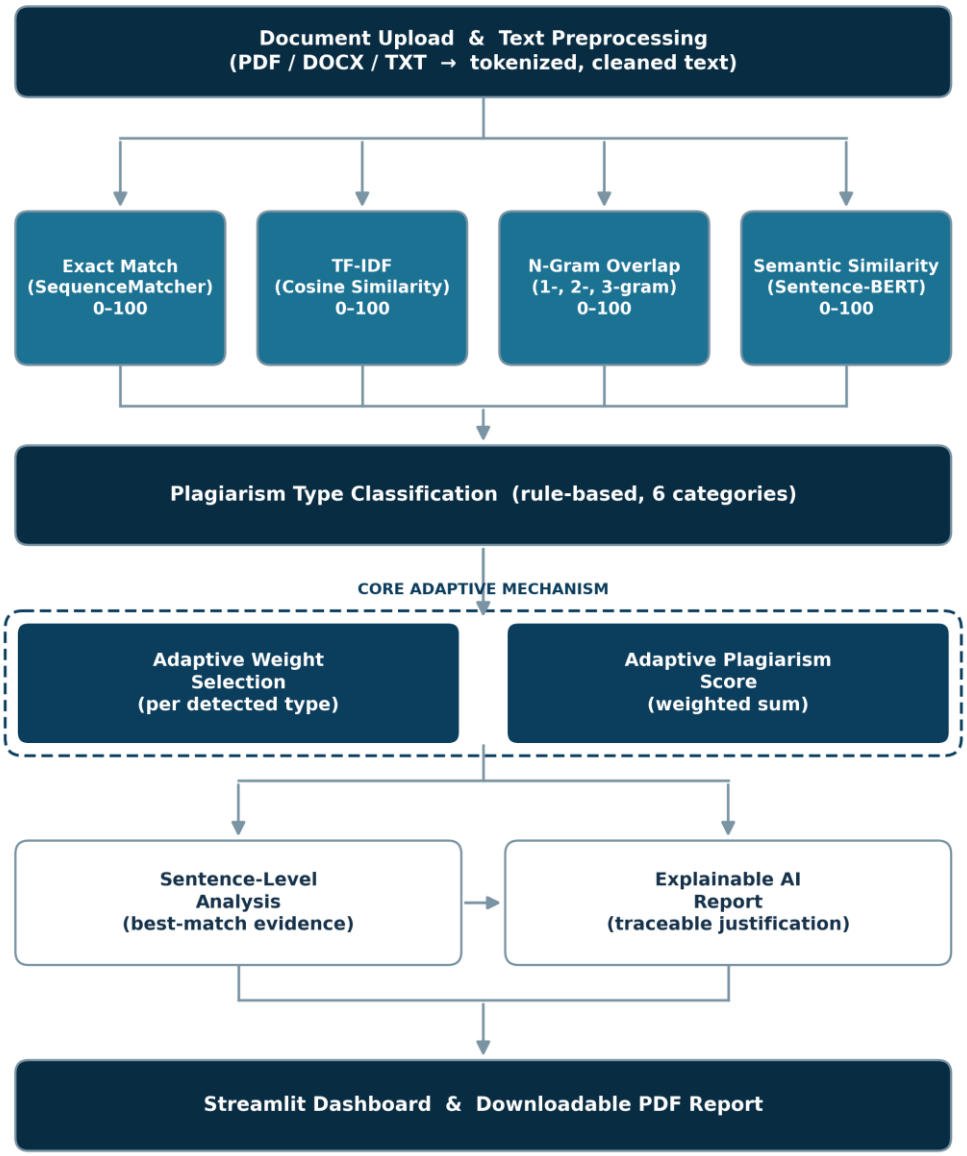


Fig.1: System Architecture

Table I. Adaptive weight profiles by detected plagiarism type

Plagiarism Type	Exact	TF-IDF	N-Gram	Semantic
Direct Copy	40%	30%	20%	10%
Partial Copy	30%	25%	25%	20%
Paraphrased Plagiarism	10%	20%	10%	60%
Structural Similarity	10%	20%	20%	50%
Mixed or Complex Similarity	25%	25%	20%	30%
Low / No Significant Plagiarism	25%	25%	25%	25%
Fixed baseline (traditional)	20%	25%	20%	35%

IV. Implementation

The system was built using Python 3.12, with Streamlit used as the front-end framework. The TF-IDF vectorizer and cosine similarity were implemented using scikit-learn, while NLTK was exploited for tokenization and lemmatization. Additionally, sentence-transformers was used for semantic similarity scores. Moreover, results are stored in a SQLite database, while ReportLab was used to generate the reports in PDF format. Finally, thresholds, weight vectors, and classification rules are stored separately in a YAML file, rather than being hardcoded. More specifically, each weight vector's elements sum up to 1.0, which allows for immediate rejection of invalid configurations, thus failing fast and hard, instead of potentially generating invalid results later during analysis.

The thresholds were defined heuristically. However, it is notable that difflib's SequenceMatcher works with sequences of characters, not words. As such, even unrelated English paragraphs will have a certain percentage of similarity due to the similarity of function words and phraseology. Therefore, the lowest threshold was determined by calculating the ratio of three pairs of unrelated documents. The mean value was found to be 32.7%, with individual values being 30.9%, 35.3%, and 31.8%. This value was then used as a threshold for the "Low or No Significant Plagiarism" category, as using a threshold close to 0% would result in most unrelated pairs of documents being classified as having significant plagiarism.

V. Experimental Results and Discussion

The system is validated in 2 ways, an automated unit-test suite exercising individual components and the controlled document-level experiments comparing adaptive and fixed-weight scorings.

A. Unit-level validation

A suite of 25 automated tests (10 of which tested the four detectors in isolation, 7 – the classification-rule behavior, and 8 – the adaptive weighting engine and hybrid scorer) passed successfully. Among the individual tests, one is of specific interest as it creates a paraphrase-shaped score distribution (exact = 15, TF-IDF = 25, n-gram = 15, semantic = 90) and checks that the adaptive score with the Paraphrased Plagiarism weight vector indeed places more emphasis on the semantic similarity than the fixed baseline would, thus proving that the adaptive mechanism favors different scoring distributions. Finally, a similar test was made with the Direct Copy distribution, where all four detectors' scores were equalized, which showed that adaptive and fixed scoring differ by no more than 5 points in such a scenario.

B. Document-level evaluation

Table II reports results for four evaluation documents, each compared against a common reference document: a near-verbatim copy, a document interleaving verbatim and original sentences, a substantially reworded paraphrase, and an unrelated document on a different topic.

Table II. Detector scores and classification outcomes across evaluation documents

Document	Exact	TF-IDF	N-Gram	Semantic	Classified Type	Adaptive	Fixed	Δ
Direct Copy sample	100.0	88.8	100.0	100.0	Direct Copy (99.0%)	96.6	97.2	-0.6
Partial Copy sample	60.0	32.9	42.4	50.8	Partial Copy (85.5%)	47.0	46.5	+0.5
Paraphrased sample	43.4	35.6	15.2	36.6	Mixed / Complex (62.0%)	33.8	33.5	+0.3
Low-similarity sample	31.5	2.4	1.7	11.1	Low / No Plagiarism (89.5%)	11.7	11.1	+0.6

The Direct Copy sample was correctly classified with high confidence (99.0%); its adaptive and fixed scores are almost identical (respectively, 96.6% and 97.2%), as would be expected given all four detectors already agree strongly, leaving little room for re-weighting to change the outcome. The interleaved-content sample was correctly classified as Partial Copy (85.5% confidence), consistent with its intermediate exact-match score (60.0%) combined with moderate n-gram and semantic overlap. The unrelated document was correctly classified as Low or No Significant Plagiarism (89.5% confidence) and received a low adaptive score (11.7%), correctly avoiding a false positive despite the 31.5% exact-match baseline noise discussed in Section IV.

The substantially reworded paraphrase sample was classified as mixed or complex similarity (62.0% confidence) instead of Paraphrased Plagiarism. This reflects a known limitation of the evaluation environment rather than the classification logic itself, semantic scoring for this run used the character n-gram fallback described in Section III, because the full Sentence-BERT model requires a one-time internet-dependent download that was unavailable in the evaluation environment. The fallback captures shared character sequences and word roots but not deep meaning equivalence, and consequently under-estimates semantic similarity for genuine paraphrases compared to the full transformer model. We report this result transparently as a limitation of the offline fallback path, and expect — consistent with the semantic detector's behaviour on identical and near-identical sentences during unit testing — that evaluation with the full Sentence-BERT model would raise the semantic score sufficiently to trigger the intended Paraphrased Plagiarism classification. Validating this directly on a larger corpus using the full model is identified as future work.

VI. Conclusion and Future Work

This paper presentes a unique adaptive plagiarism detection system that classifies likely types of textual overlaps before weighing four independent measures of similarity with varying weights, rather than applying a global fixed formula on all the documents. Automated testings confirms that this adaptive system yields significantly different scores specifically where detection methods disagree on content, such as paraphrasing, and converge with a traditional fixed-weight baseline where detection methods already agree, such as direct copying. Document-level evaluations on four representative cases correctly identified direct copying, partial copying, and unrelated content with high confidence; evaluating substantially reworded paraphrases with the full Sentence-BERT model, rather than the offline fallback as used in this evaluation environment, is identified as an immediate future work. future directions

include evaluating on a larger, publicly-available plagiarism corpus (e.g., PAN plagiarism detection datasets), extension to languages besides English, and a user study evaluating whether the system's generated explanations improve reviewer trust and review speed compared to a bare similarity.

Acknowledgment

The authors would like to thank Mrs. Deepti N N for her guidance and supervision of this project at Rajiv Gandhi Institute of Technology, Bengaluru, under the Visvesvaraya Technological University Major Project Phase II curriculum (BCSP786).

References

[1] A. Aiken, "MOSS: A System for Detecting Software Plagiarism," Stanford University, 1994.

[2] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in Proc. 2014 Conf. Empirical Methods Natural Lang. Process. (EMNLP), Oct. 2014, pp. 1532-1543.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. 2019 Conf. Empirical Methods Natural Lang. Process. (EMNLP), Nov. 2019, pp. 3982-3992.

[4] P. Saadatpanah, A. Shafahi, and T. Goldstein, "Adversarial attacks on copyright detection systems," in Proc. 2nd Workshop Adversarial Machine Learning in Real-World Computer Vision Applications, Jun. 2019, pp. 1-8.

[5] Qamar, Z. Ali, S. Malik, A. Q. Khan, and M. Ahmar, "AI-Based Plagiarism Detection System," International Journal of Innovations in Science & Technology, vol. 7, no. 10, pp. 184–189, Dec. 2025.

[6] F. Shaikh and S. Siddavatam, "AI-Based Plagiarism Detection System," International Journal of Engineering Research & Technology (IJERT), vol. 14, no. 2, pp. 1–5, 2026.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[8] W. B. Cavnar and J. M. Trenkle, "N-Gram-Based Text Categorization," in *Proc. 3rd Annual Symposium on Document Analysis and Information Retrieval (SDAIR)*, 1994, pp. 161–169.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[10] M. A. Alzahrani, V. Palade, and N. Salim, "On retrieving intelligently plagiarized documents using semantic similarity," *Engineering Applications of Artificial Intelligence*, vol. 45, pp. 246–258, 2015.